

TRAITEMENT D'IMAGES SPORTIVES ET MÉCATRONIQUE AU SERVICE DE LA CÉCITÉ

LYLIA MESA - 34247

TRAITEMENT D'IMAGES SPORTIVES ET MÉCATRONIQUE AU SERVICE DE LA CÉCITÉ

Accessibilité des matches pour les personnes malvoyantes



Audiodescription pour les spectateurs malvoyants (Coupe du Monde de rugby, 2023)

© Ilian Valet



Touch2See

© Touch2See

TRAITEMENT D'IMAGES SPORTIVES ET MÉCATRONIQUE AU SERVICE DE LA CÉCITÉ

Etude de la position d'une balle



Balle équipée d'une centrale inertielle
(Coupe du Monde de football, 2022)

© Adidas



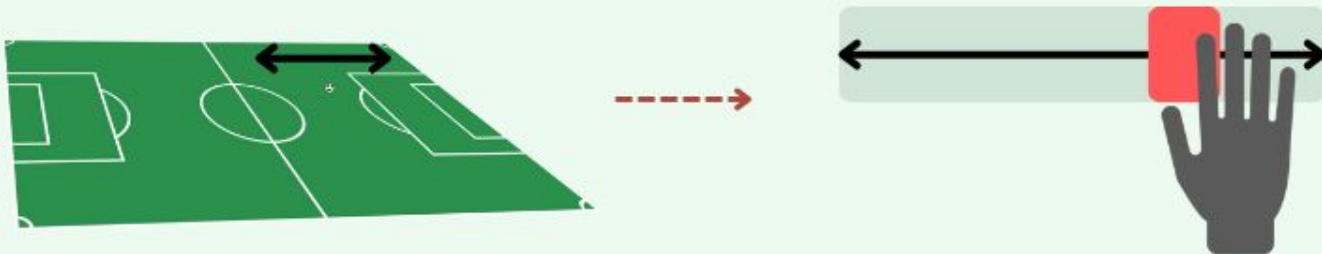
Caméras autour d'un terrain de football
(Finale de la Ligue des Champions, 2023)

© Dan Chung

Enjeux :

— Etude d'un système visant à :

- Recueillir les **positions de la balle** sur un terrain de football à partir d'images
- Transmettre en temps réel **l'information par le toucher**



— Évaluation des performances

— Détermination de la validité d'une approche à une caméra

SOMMAIRE

01

TRAITEMENT DE L' INFORMATION VISUELLE

- a. Principe de l'algorithme

02

TRANSMISSION DE L'INFORMATION HAPTIQUE

- a. Choix du mécanisme
- b. Mise en mouvement

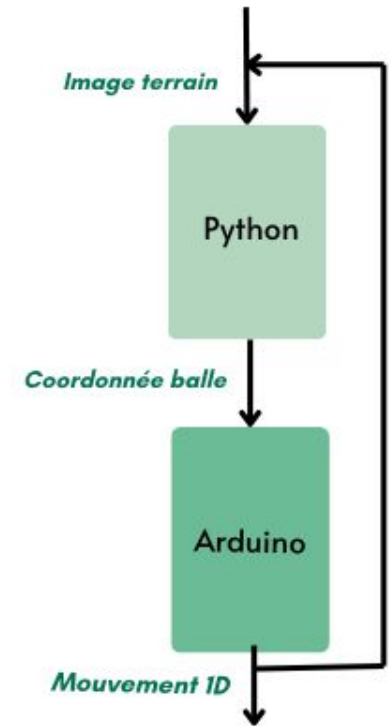
03

PERFORMANCES

- a. Algorithme
- b. Système mécanique
- c. Extension du projet

04

CONCLUSION

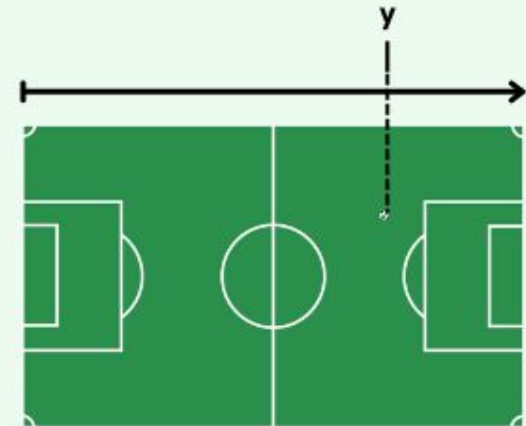


I. TRAITEMENT DE L'INFORMATION VISUELLE

**Entrée :**

vidéo = succession
d'images

Python

**Sortie :**

coordonnée y du
ballon

Hypothèses :

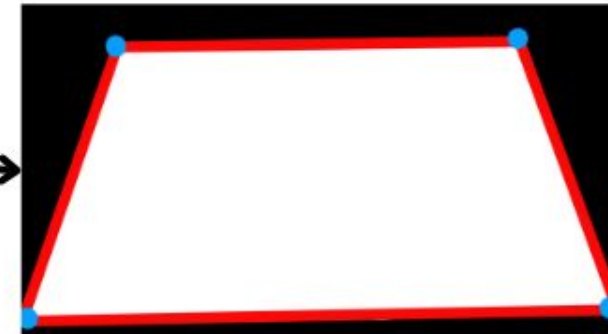
- (H1) Ballon au sol
- (H2) $\emptyset$ distorsion image
- (H3) 4 coins terrain filmés

I. TRAITEMENT DE L'INFORMATION VISUELLE

A. PRINCIPE DE L'ALGORITHME



© Pond5

Image caméraSegmentation couleur
Filtrage bruit

Contours, coins



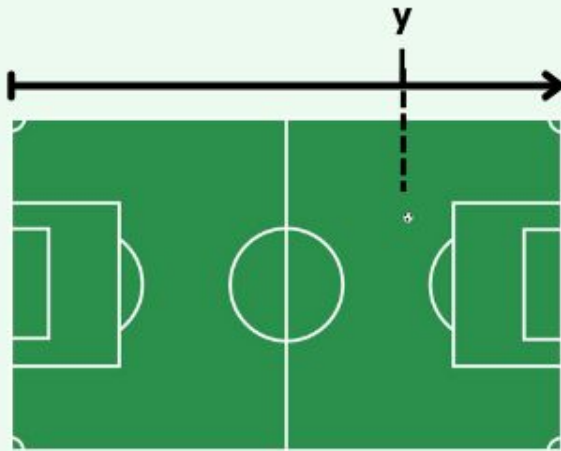
Matrice projection



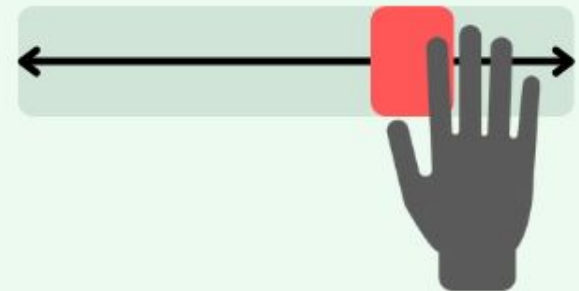
Coordonnées balle

Projection
**Coordonnées balle
corrigées**

II. TRANSMISSION DE L'INFORMATION HAPTIQUE



Entrée : succession
coordonnées y
(Python)



Sortie :
Mouvement
translation 1D

Création du mouvement ID

- Moteurs linéaires
- Moteurs rotatifs



*Moteurs à courant continu, synchrone,
asynchrone*



Pas à pas



Servomoteur

Position

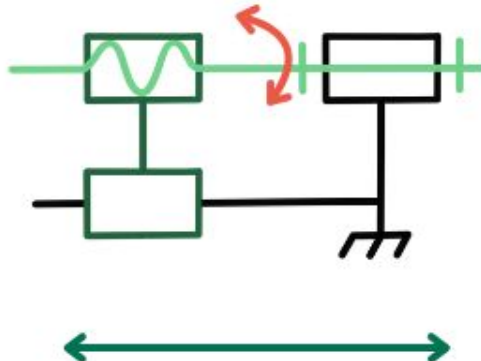
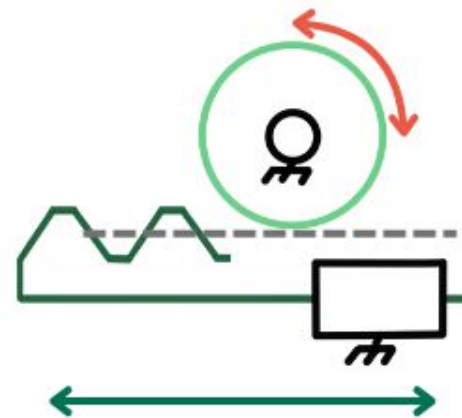
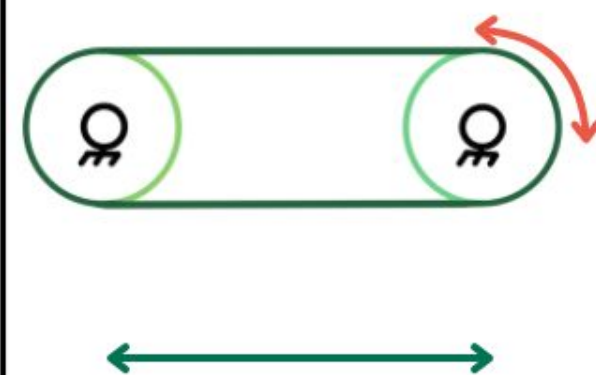
Vitesse

Moteurs rotatifs et usages

© techbriefs.com

Transformation mouvement : rotation -> translation

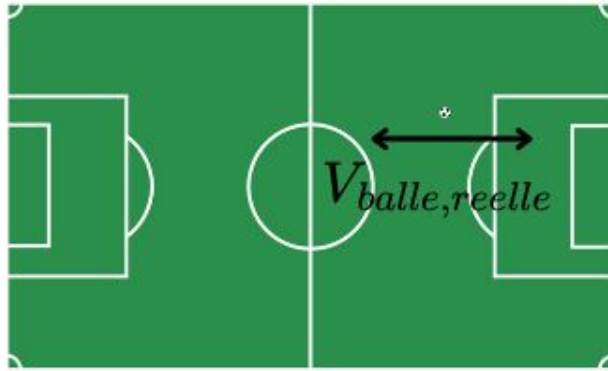
Relation linéaire vitesses rotation / translation

Vis écrou	Pignon crémaillère	Poulie courroie
		

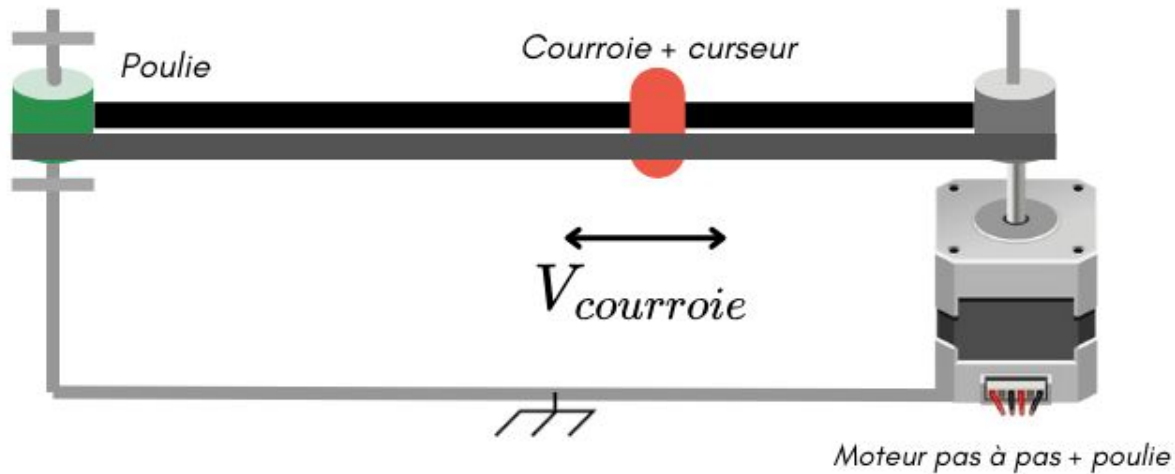
- Réversibilité
- Ø usure
- Course ajustable

II. TRANSMISSION DE L'INFORMATION HAPTIQUE

A. CHOIX DU MÉCANISME



$L_{terrain}$



$L_{systeme}$

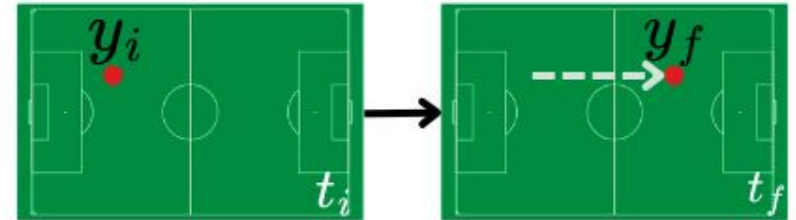
$$V_{courroie} = k \times V_{balle, reelle} : (1) \quad \text{où} \quad k = \frac{L_{systeme}}{L_{terrain}}$$

Le système doit vérifier :

$$V_{courroie} = k \times V_{balle, reelle} \quad : (1) \quad \text{où} \quad k = \frac{L_{systeme}}{L_{terrain}}$$

I - Vitesse de rotation du moteur

$$V_{balle, reelle} = \lim_{dt \rightarrow 0} \frac{dy_{balle, reelle}}{dt} \approx \frac{y_f - y_i}{t_f - t_i}$$



$$(1) \Leftrightarrow r\omega_{rot, rad} = k \frac{(y_f - y_i)}{(t_f - t_i)} \Leftrightarrow \omega_{rot, rad} = \frac{k(y_f - y_i)}{r(t_f - t_i)} \quad \text{en rad/s}$$

Arduino : vitesses en tour / min (rpm) :

$$\omega_{rot} = \lambda \frac{k(y_f - y_i)}{r(t_f - t_i)} \quad \text{où} \quad \lambda = \frac{60}{2\pi}$$

2 - Nombre de pas à réaliser

$$\omega_{rot,rad} = \lim_{dt \rightarrow 0} \frac{d\theta_{poulie}}{dt} \approx \frac{(\theta_f - \theta_i)}{(t_f - t_i)}$$

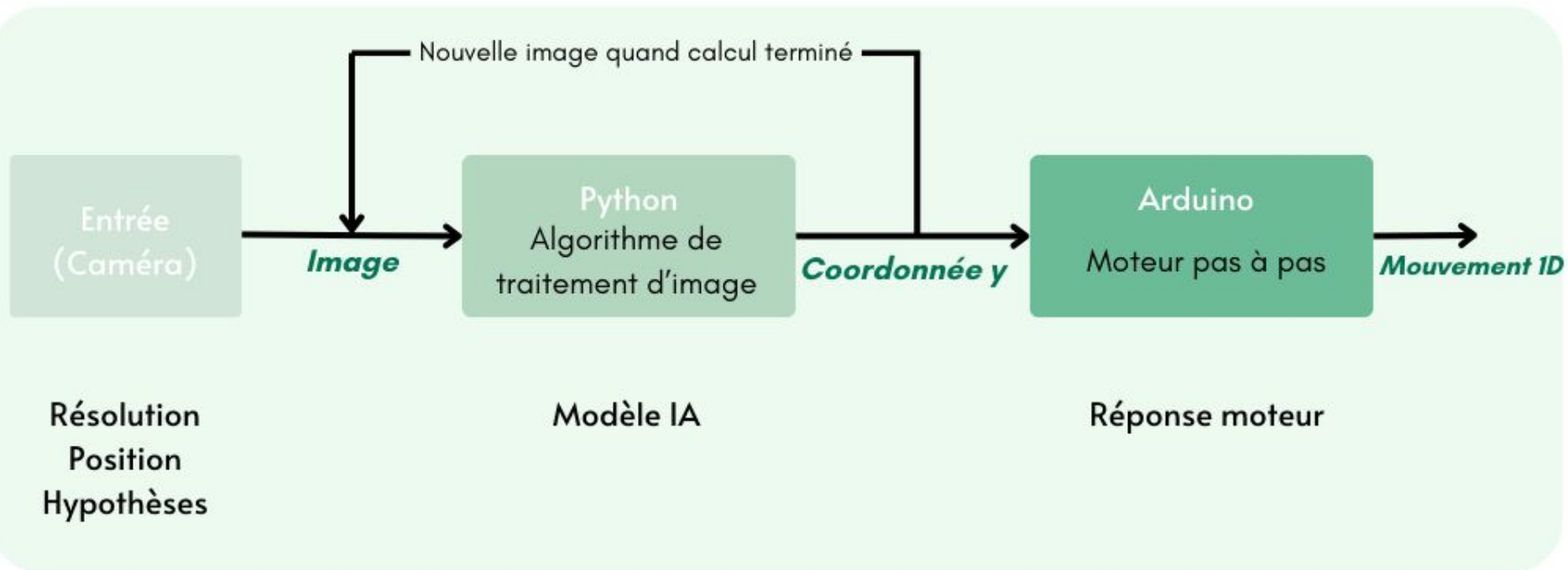
$$\Rightarrow \frac{\theta_f - \theta_i}{t_f - t_i} = \frac{k(y_f - y_i)}{r(t_f - t_i)} \Rightarrow \theta_f - \theta_i = \frac{k(y_f - y_i)}{r}$$

$$\text{Or } \theta_f - \theta_i = N_{pas} \theta_{1pas}$$

$$N_{pas} = \frac{k(y_f - y_i)}{r\theta_{1pas}}$$

III. PERFORMANCES DU SYSTÈME

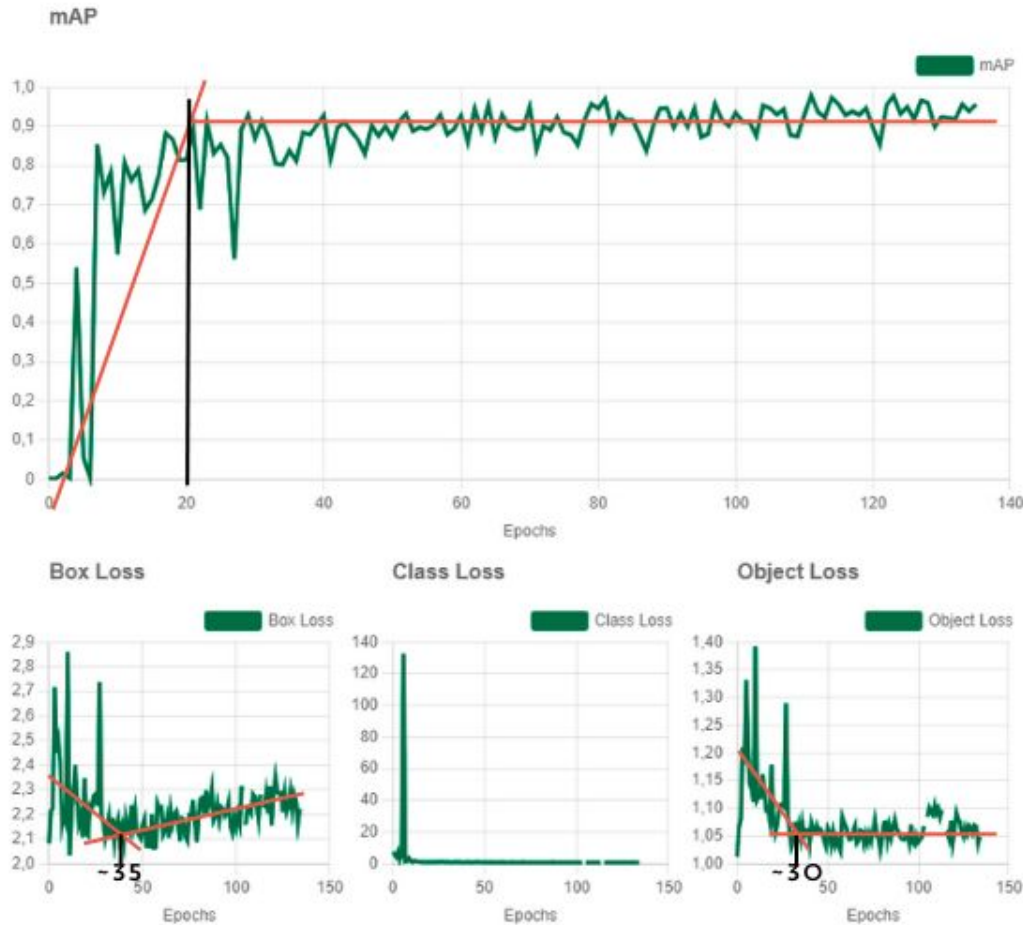
Etude de la rapidité et de la précision



III. PERFORMANCES DU SYSTÈME RÉEL

A. ALGORITHME

I - Modèle de l'IA



35 epoch

Courbes d'entraînement du modèle
(interface d'entraînement Roboflow)

2 - Paramètres d'entrée

Logiciel de CAO pour générer images terrains / ballons :

Paramètres fixes

**Positions balle,
caméra**

→ **Grande quantité de tests individuels avec les mêmes conditions expérimentales**

2 - Paramètres d'entrée : Résolution

Blender

230 positions balle
/ résolution
Caméra fixée



Python

Distance
Temps calcul
Moyennes/résolut°



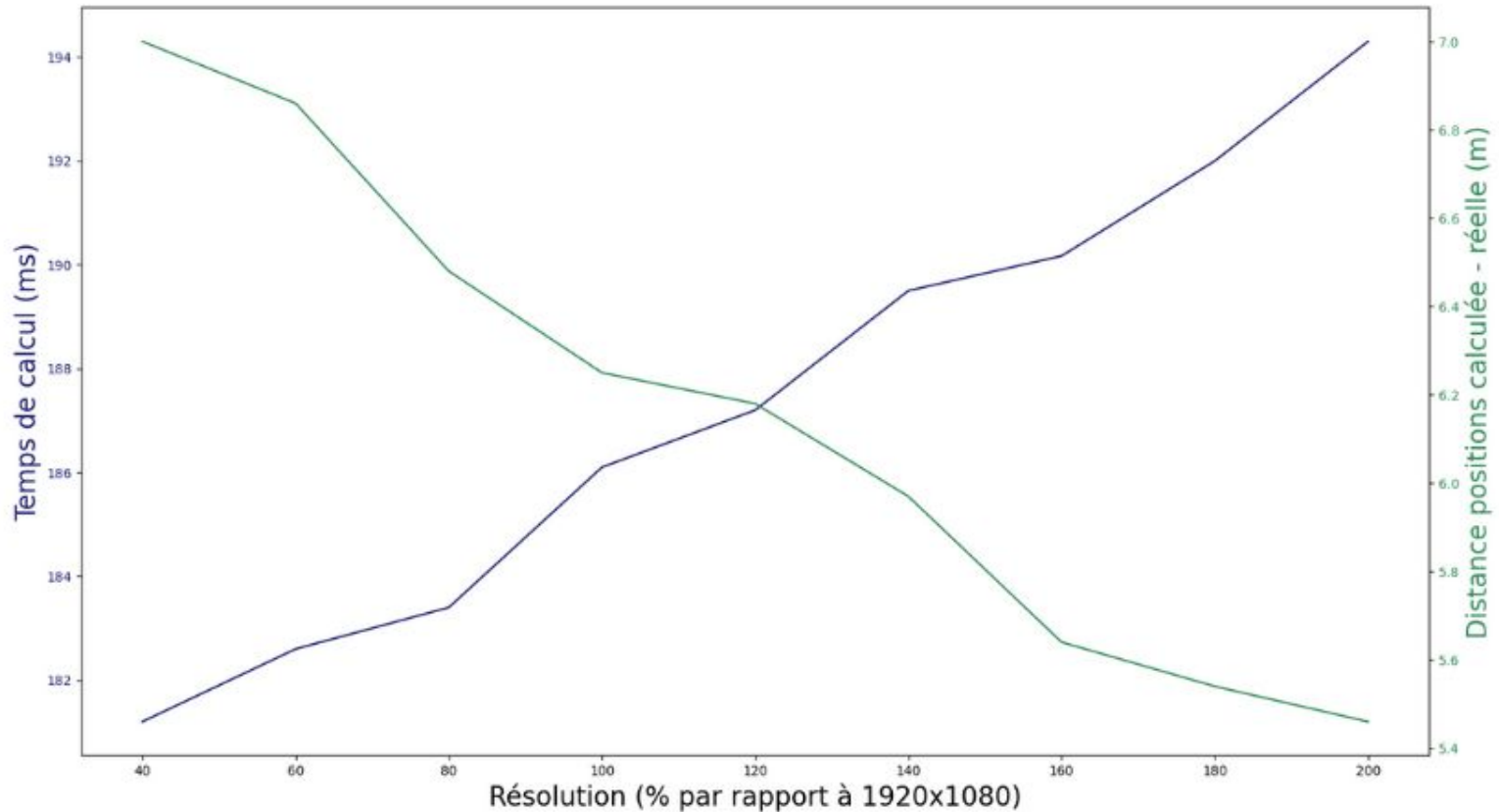
Exemples d'images générées

Distance(m)

$$\delta = |y_{exp} - y_{th}|$$

2 - Paramètres d'entrée : Résolution

-> Compromis vitesse précision

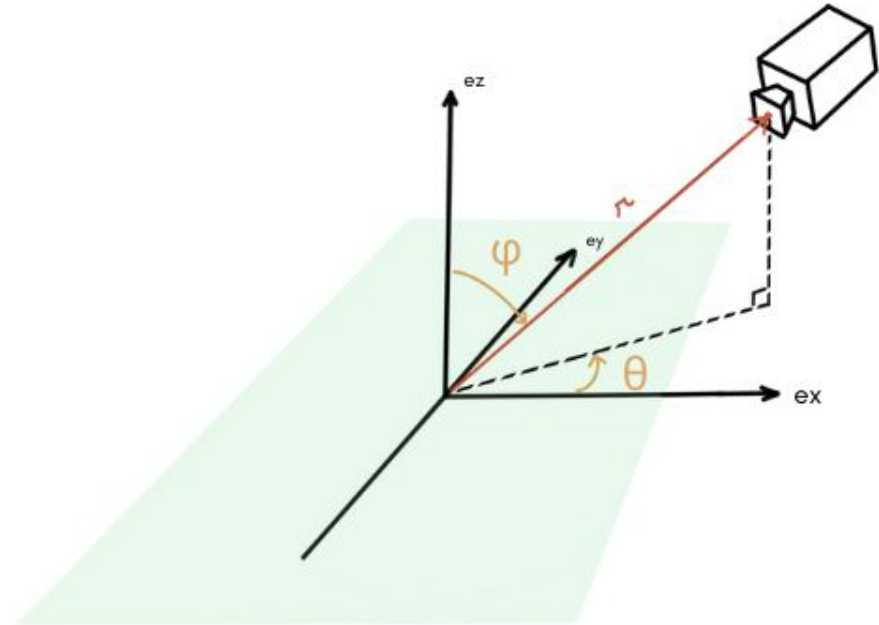


Temps de calcul et distance position calculée / réelle en fonction de la résolution

80 %

2 - Paramètres d'entrée : Position de caméra

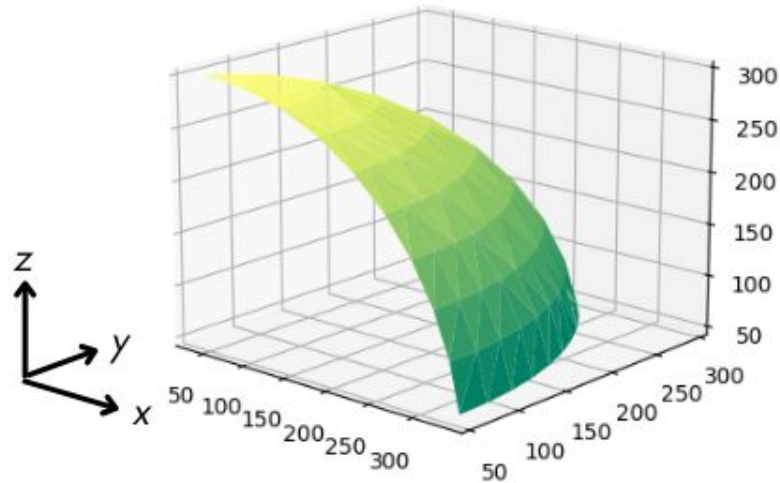
- **Élévation (φ) et azimut (θ)**
- Coordonnées sphériques (r, θ, φ) , r fixé
- origine repère : centre terrain



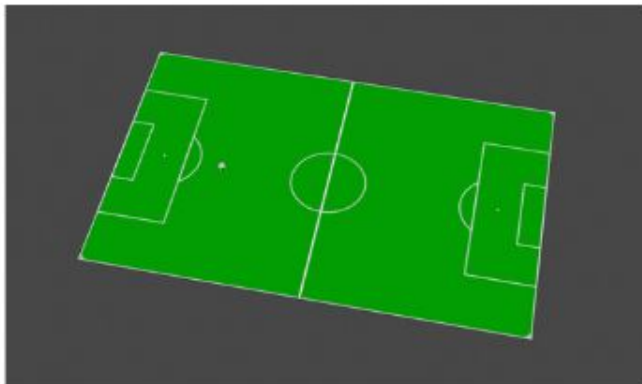
Positionnement d'une caméra dans l'espace

Dans la suite : étude des positions sur 1/4 du terrain (symétries)

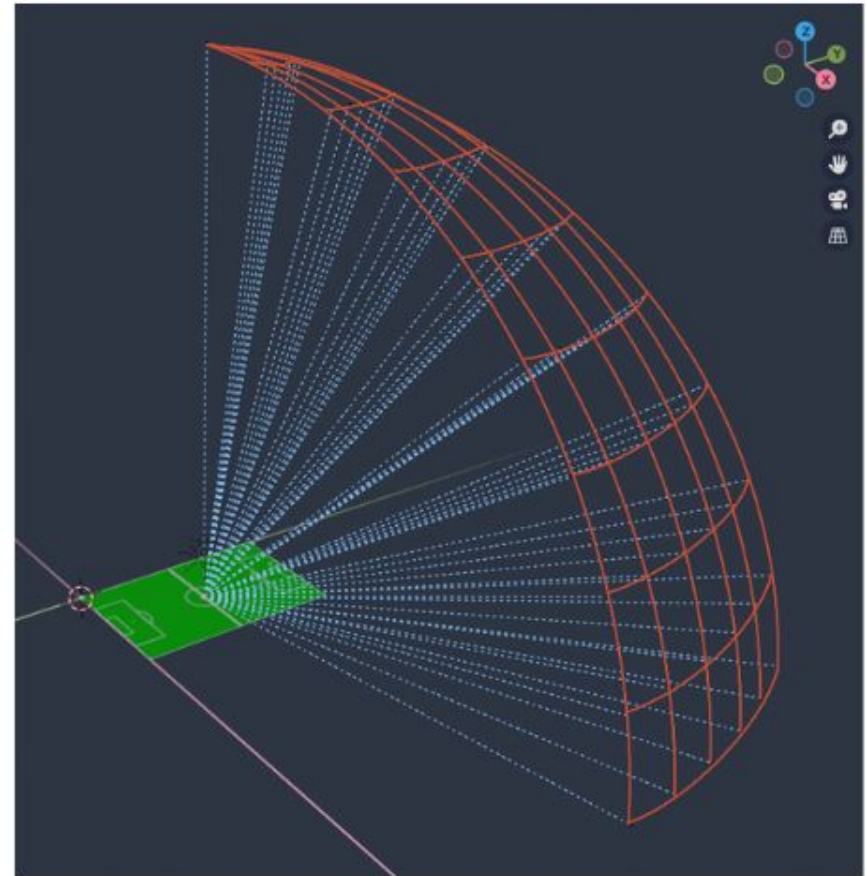
2 - Paramètres d'entrée : Position de caméra



Représentation visuelle des coordonnées
générées sur Python ($r=300\text{m}$)

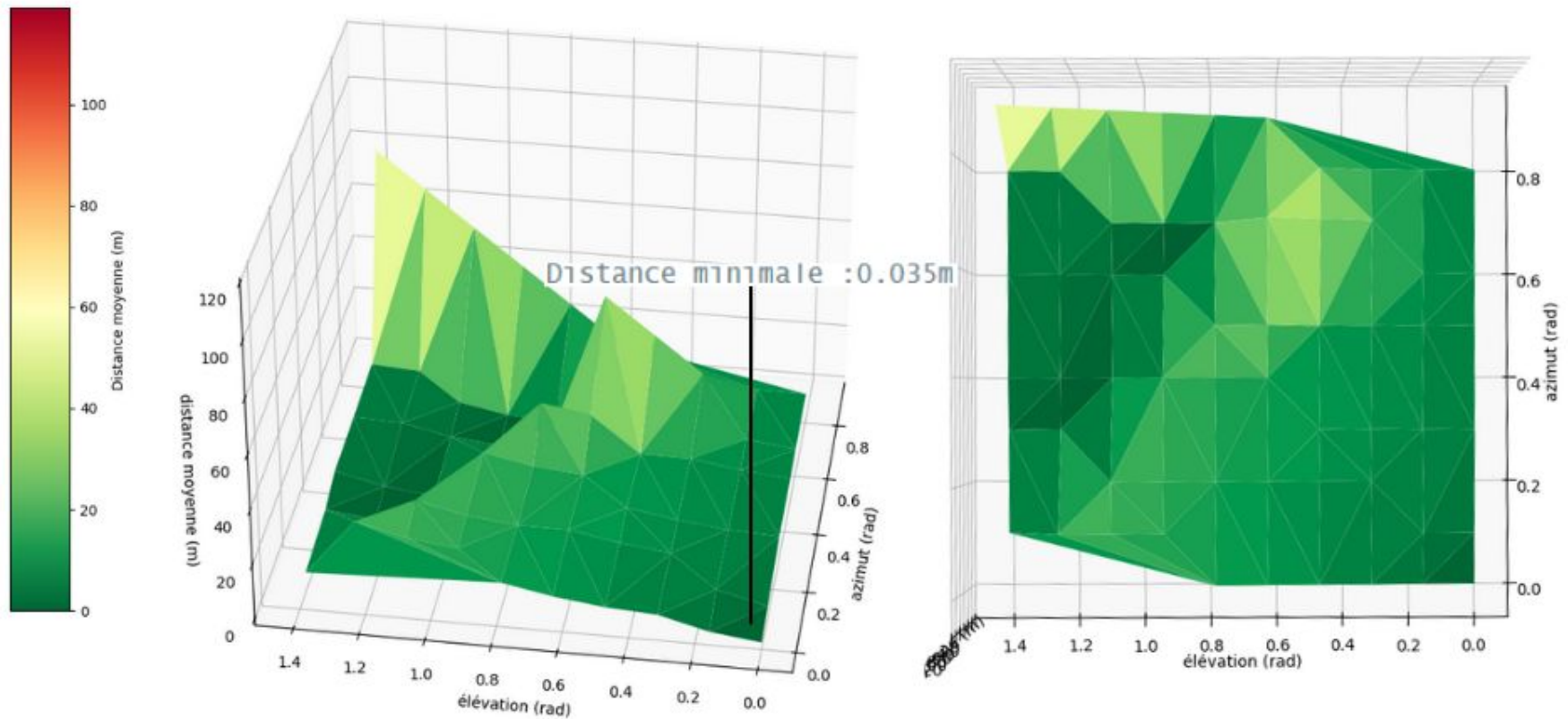


Exemple d'image générée



Interface Blender - 60 caméras placées aux
coordonnées générées

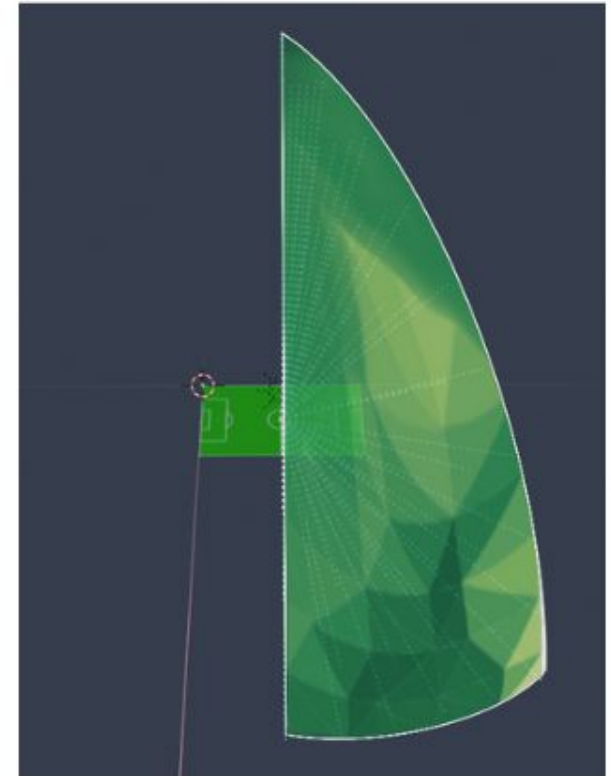
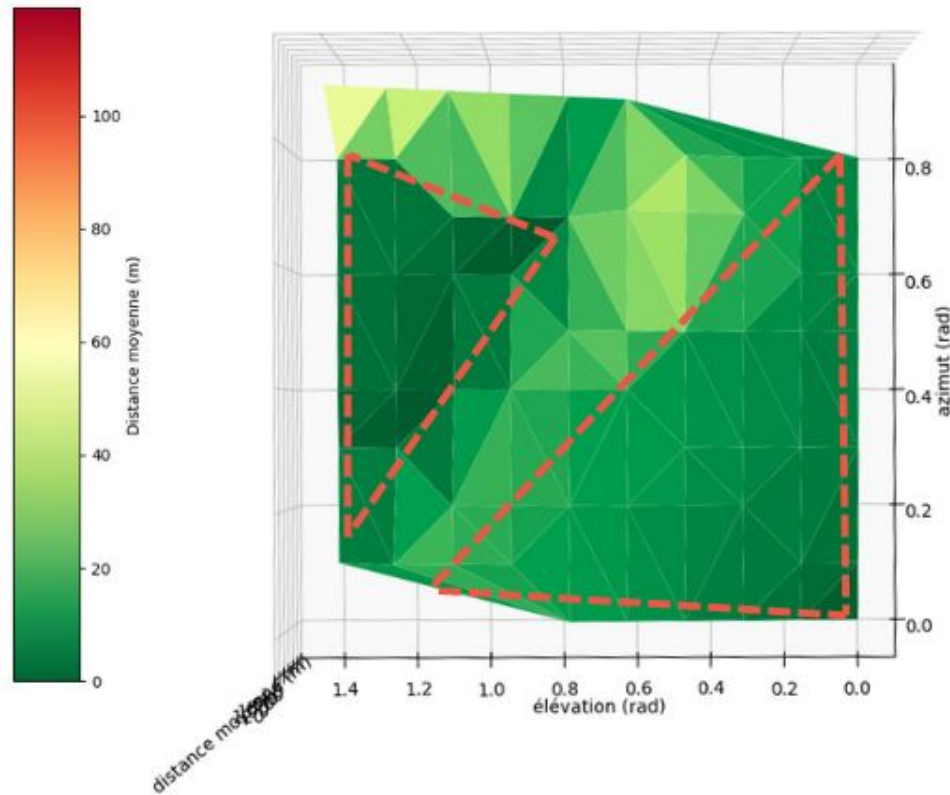
2 - Paramètres d'entrée : Position de caméra



Distance moyenne positions calculée / réelle en fonction de l'azimut et
l'élévation de la caméra

60 caméras, moyenne sur 9 positions de balle

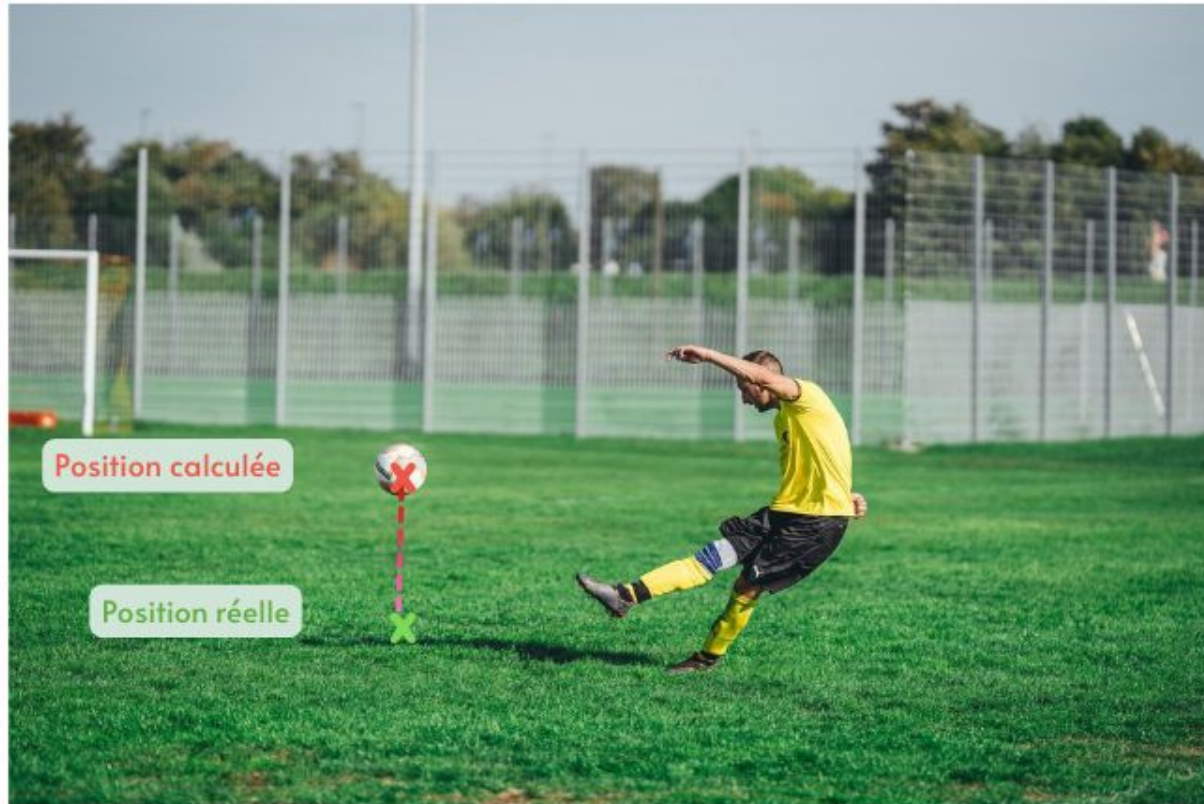
2 - Paramètres d'entrée : Position de caméra



Distance moyenne positions calculée / réelle en fonction de l'azimut et l'élévation de la caméra

$\varphi, \theta \rightarrow 0,0$ ou $\varphi \rightarrow 90^\circ$

3 - Hypothèse : “Balle au sol”

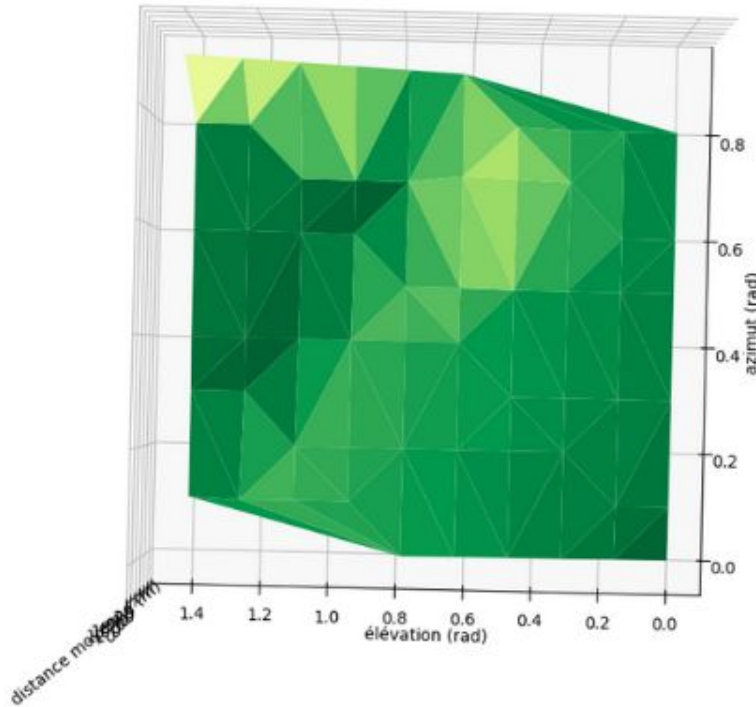


Erreur liée à la hauteur de la balle et le positionnement de la caméra

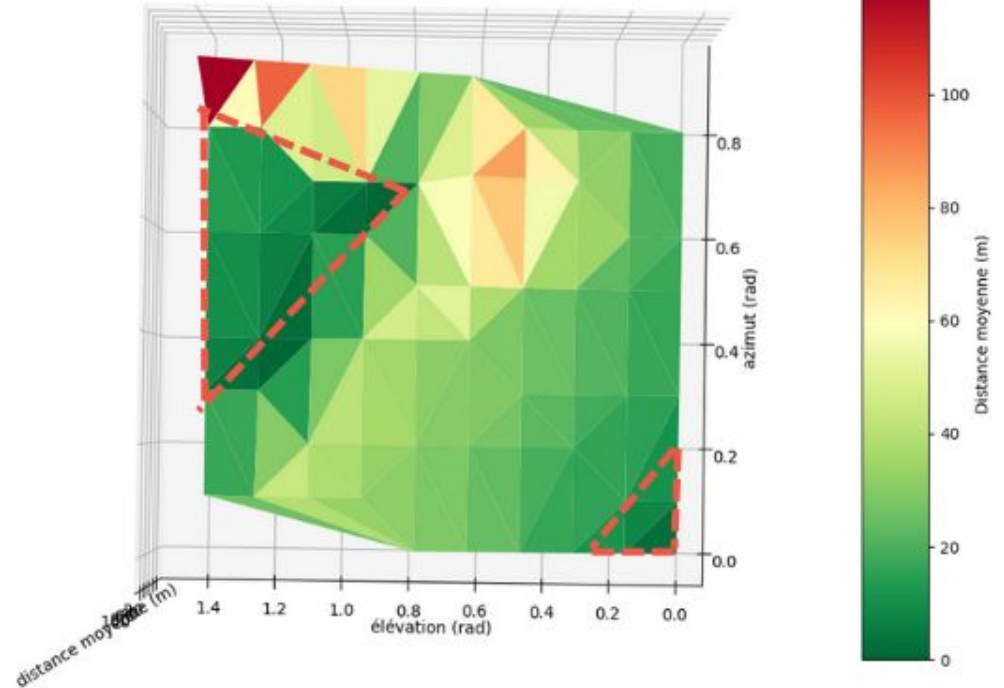
© Open Goaaal USA

3 - Hypothèse : “Balle au sol”

Balle au sol



Hauteur balle variable

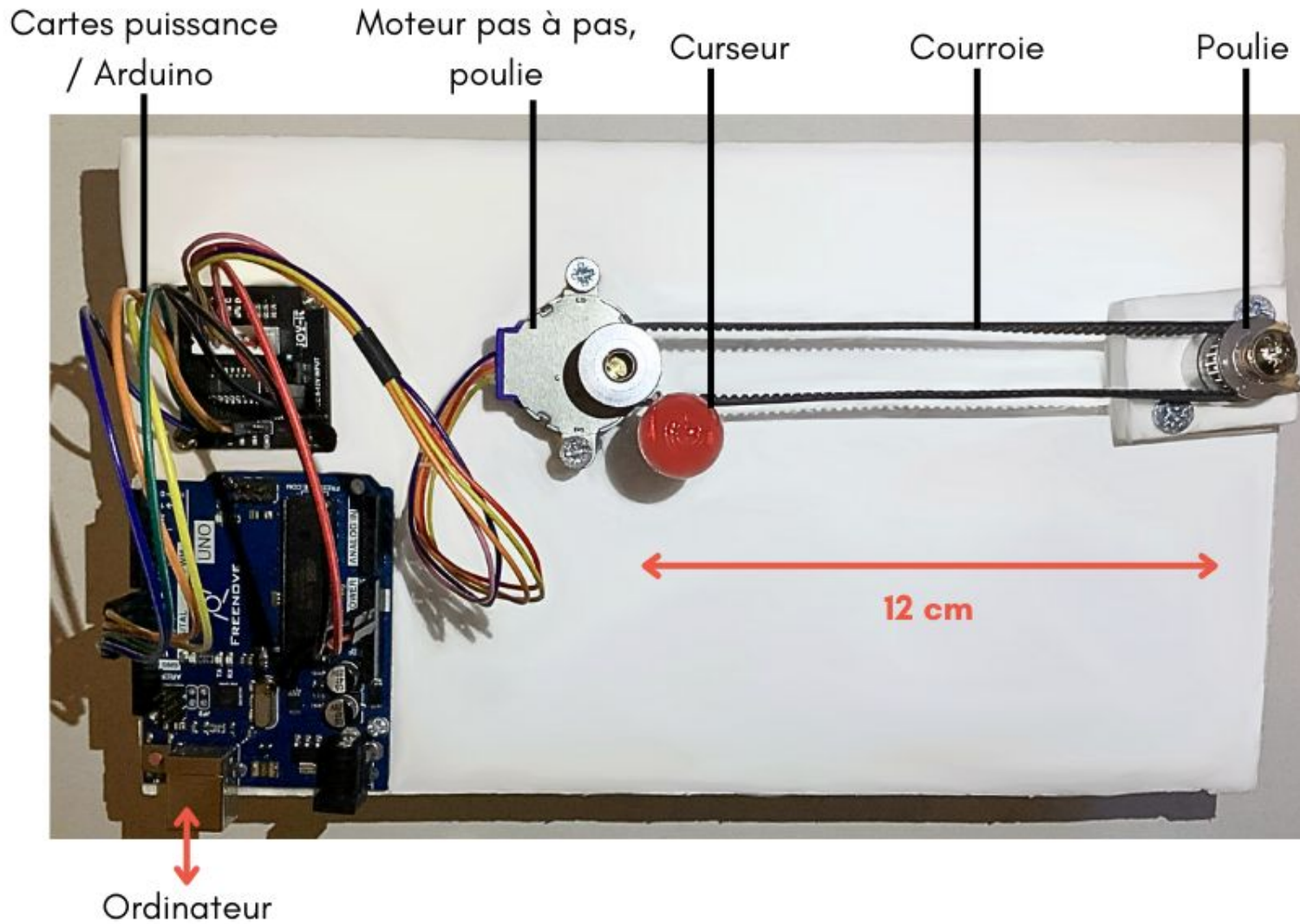


Comparaison : distances moyennes en fonction de l'azimut et l'élévation de la caméra

Hypothèse non nécessaire à certaines positions de caméra

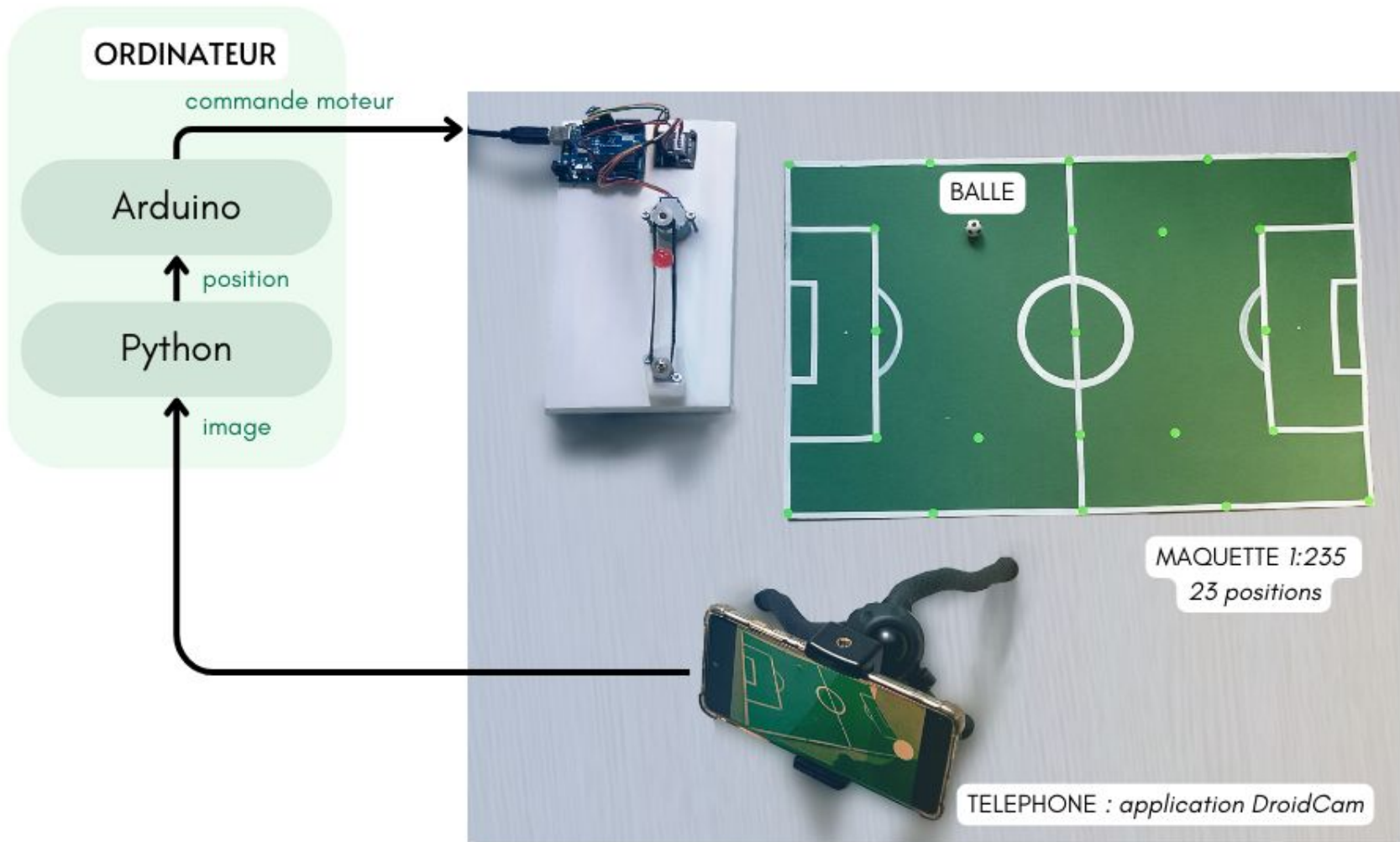
III. PERFORMANCES DU SYSTÈME RÉEL

B. SYSTÈME MÉCANIQUE



Montage poulie-courroie

I - Précision (système entier)



Expérience de pointage

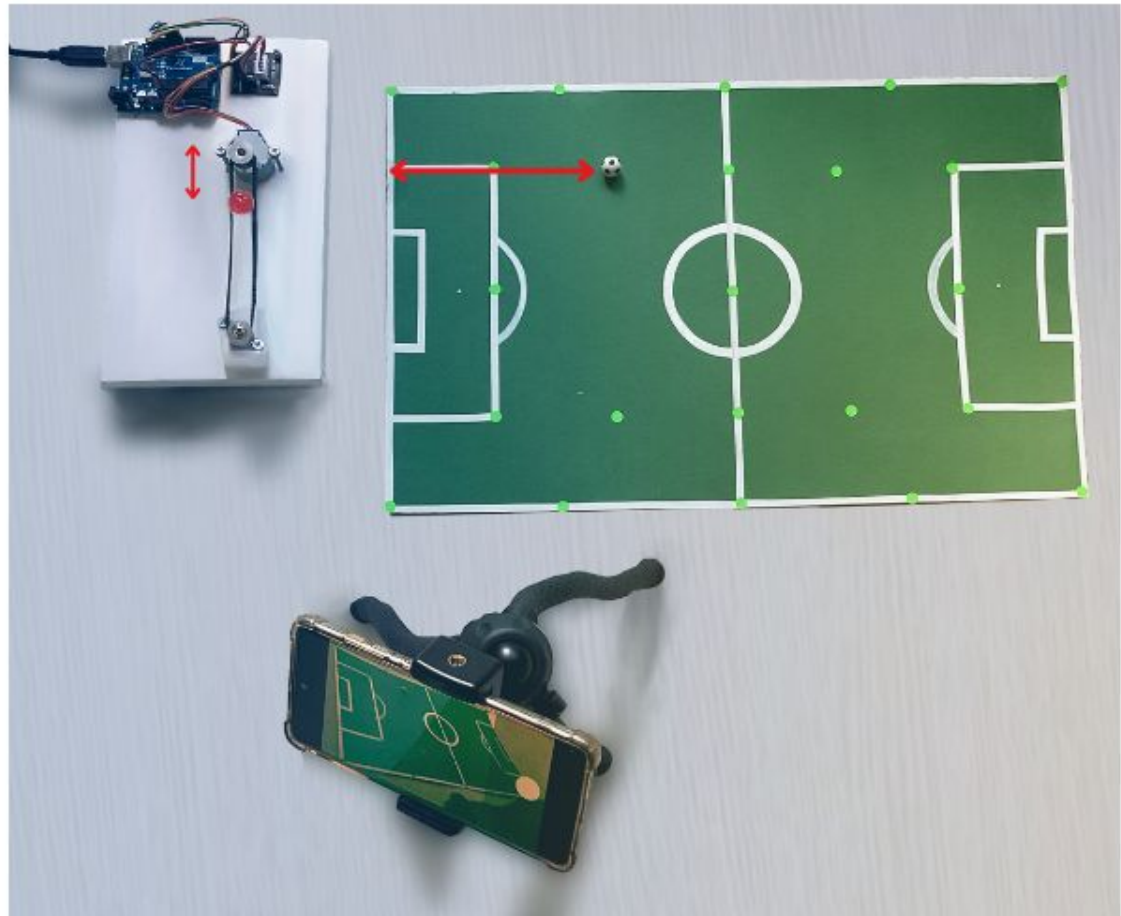
I - Précision

Erreur moyenne :

$$e = 1.6 \pm 0.5 \text{ cm}$$

Sources :

- Algorithme
- Poulie-courroie
- Mesures



II - Rapidité : Vitesse

Arduino : rotation
vitesse variable
> Vitesse max
→ arrêt

Résultats :

$$\omega_{\max} = 31844 \text{ pas/min}$$

$$\begin{aligned}\omega_{\max} &\approx 15,5 \text{ tour/min} && (2048 \text{ pas/tour}) \\ &\approx 1.62 \text{ rad/s}\end{aligned}$$

Or
$$V_{balleMax} = \frac{\omega_{rot} \times r}{k}$$

$$V_{balleMax} = 31.2 \text{ kmh}^{-1}$$

II - Rapidité : Vitesse

$$V_{balleMax} = 31.2 kmh^{-1}$$

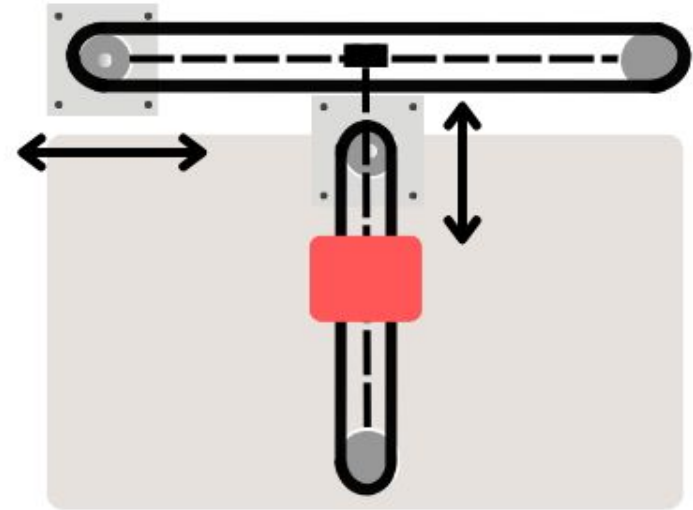
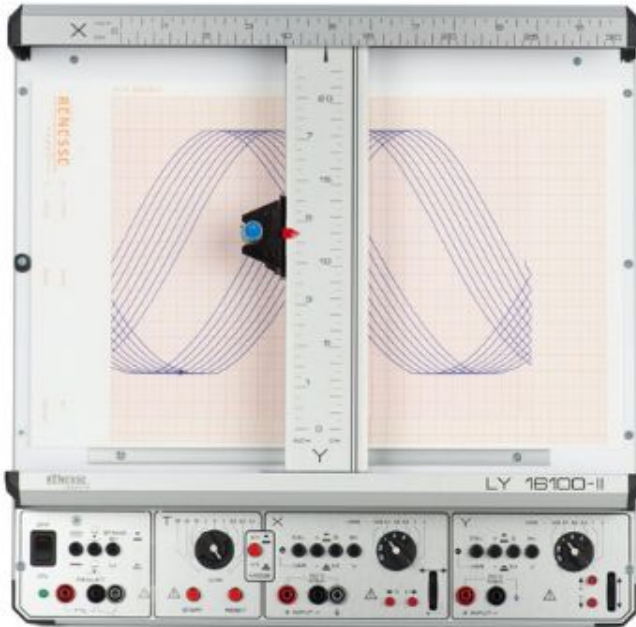
Résultat insuffisant :

$$V_{balleMoy} = 85 kmh^{-1}$$

Solutions :

- Système plus petit
- Moteur plus performant (ex NEMA17)

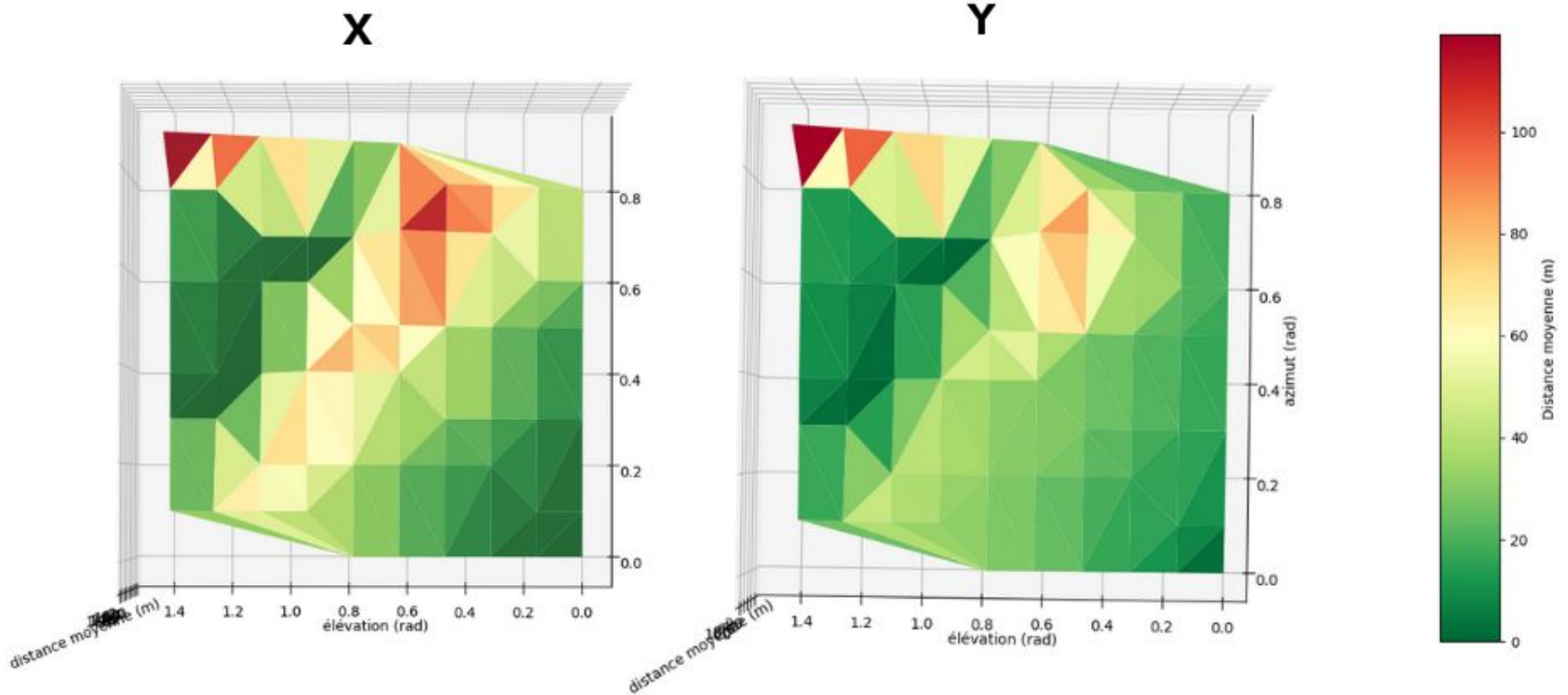
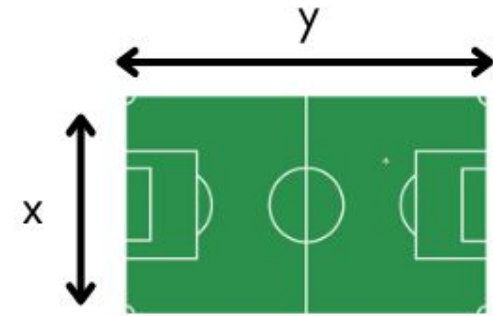
Transmettre un mouvement 2D



Principe d'une table traçante

photo : © kp8.fr

Limite de l'algorithme à une caméra



Distance moyenne en fonction de l'azimut et l'élévation de la caméra

IV. CONCLUSION

	Modèle IA	Résolution (entrée)	Position caméra	Améliorations
Rapidité	/	⊖ vidéo < 5 fps	/	Moteur plus performant
Précision	35 epoch	⊕	$\theta, \varphi \rightarrow 0^\circ, 0^\circ$ $\varphi \rightarrow 90^\circ$	2+ caméras (stéréovision)
Stabilité	/	/	/	/

ANNEXE

P. 34 Algorithmes

P. 46 Principe de l'algorithme

- Filtrage du bruit - seuillage
- Détection de contours
- Approximation contours
- Détection de coins
- Calcul d'une matrice de projection

P. 52 Réseau de neurones

- Principe
- Entraînement du modèle utilisé
- Evaluation de la précision

P. 55 Electronique

- Montage
- Moteur pas à pas

P. 57 Stéréovision

P. 58 Complexité

P. 59 Bibliographie

ALGORITHMES- PYTHON BOUCLE PRINCIPALE

```

1  ## Modules et données
2
3  import cv2
4  import numpy as np
5
6  # Modèle de détection
7  from inference.models.utils import get_roboflow_model
8  model_name = "balles-de-football"
9  model_version = "1"
10 model = get_roboflow_model(model_id="{}/{}".format(model_name, model_version), api_key="WuPJ2uBzqVxnodwmTxCu")
11
12 # Accès à la caméra
13 video = cv2.VideoCapture(0)
14 assert video.isOpened(), "image inaccessible"
15
16 # Segment de couleur
17
18 vert1 = np.array([0, 0, 100])
19 vert2 = np.array([100, 255, 255])
20
21
22 # Coordonnées des points pour la projection
23 destination = np.array([(0, 0), (0, 119), (75, 0), (75, 119)], dtype=np.float32)
24
25 # Données pour la détection du terrain
26 nb_coins = 4
27 qualite = 0.1
28 aire_min = 100000
29
30 ## Connexion avec Arduino
31
32 import serial
33 arduino = serial.Serial(port='COM3', baudrate=115200, timeout=.1)
34
35 def envoi_coordonnee(x):
36     arduino.write(bytes(x, 'utf-8'))
37
38
39 ## Boucle principale : vraie tant qu'il y a une image en entrée; jusqu'à Echap
40
41 while True and cv2.waitKey(1) != 27 :
42
43     # Lecture de chaque image de la vidéo
44     ret, frame = video.read()
45     if not ret:
46         break
47
48     # 1er filtre : segmentation par couleur
49     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
50     masque = cv2.inRange(hsv, vert1, vert2)
51     img_masque = cv2.bitwise_and(frame, frame, mask=masque)
52
53     # 2eme filtre : bruit
54     img_gris = cv2.cvtColor(img_masque, cv2.COLOR_BGR2GRAY)
55     noyau = np.ones((10, 10), np.uint8) # = matrice de convolution

```

```

56 seuil = cv2.threshold(img_gris, 127, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)[1]
57 seuil = cv2.morphologyEx(seuil, cv2.MORPH_CLOSE, noyau) #dilatation - erosion (fonction "closing")
58
59
60 #Recherche TERRAIN
61
62 # Image binarisée des contours (algorithme Canny)
63 bords = cv2.Canny(seuil, 100, 200)
64
65 # Listes des vecteurs et des hierarchies (liens parents-enfants) des contours dessinés
66 # Vecteur = liste de points (x,y)
67 contours, hierarchie = cv2.findContours(bords, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
68
69 c = []
70
71 for contour in contours:
72     approx = cv2.approxPolyDP(contour, 3, True) #lissage du contour (algorithme Ramer Douglas Peucker)
73     rectangle = cv2.boundingRect(approx) #cadre rectangulaire autour du contour -> approximation aire (=> filtre de taille)
74     L,h = rectangle [2], rectangle [3]
75     aire = L * h
76
77     if aire > aire_min:
78
79         enveloppe = cv2.convexHull(contour) #enveloppe convexe : plus petite enveloppe contenant tous les points du contour
80         cv2.polylines(frame, [enveloppe], True, (0, 0, 255), 2) #tracé de l'enveloppe
81
82         hauteur, largeur = bords.shape[:2]
83         enveloppe_img = np.zeros((hauteur, largeur), dtype=np.uint8)
84         cv2.drawContours(enveloppe_img, [enveloppe], 0, 255, 2) #tracé de l'enveloppe convexe
85
86         # détection des coins (algorithme Shi Tomasi)
87         distance_min = int(max(hauteur, largeur) / nb_coins)
88         coins = cv2.goodFeaturesToTrack(enveloppe_img, nb_coins, qualite, distance_min)
89         coins = np.intp(coins)
90
91         for coin in coins:
92             x, y = coin.ravel()
93             c.append((x, y))
94
95

```



```

96 if len(c) == nb_coins: #-4
97
98     #tri des points d'intérêt par y croissants puis x croissants par paires (correspondance pour projection)
99     coins_tril = sorted(c, key = lambda x: x[0])
100     coins = sorted(coins_tril[:2], key = lambda x: x[1]) + sorted(coins_tril[2:4], key=lambda x: x[1])
101
102
103     # PROJECTION
104
105     source = np.array(coins,dtype=np.float32)
106     # calcul de la matrice de projection H (3x3)
107     H, _ = cv2.findHomography(source, destination)
108
109
110     #DETECTION BALLE
111
112     results = model.infer(frame) #renvoie predictions 'confiance' décroissante -> premier set de coordonnées. Je suppose qu'il y
a toujours une prédiction tant que la projection est possible
113
114     #(indilage dépend de structure résultat)
115     r = str(results[0])
116     r2 = r.split(" ")
117
118     coord = str(r2[5:9])
119
120     xywh = coord[39:-1]
121     coo_boite = xywh.split(',')
122     x = float(coo_boite[0][2:])
123     y = float(coo_boite[1][2:])
124     w = float(coo_boite[2][6:])
125     h = float(coo_boite[3][7:])
126
127     coo_balle_img = np.array([x + w / 2, y + h, 1])
128
129     coo_balle = np.dot(H, coo_balle_img)
130
131
132     #ENVOI A ARDUINO
133
134     envoi_coordonnee(coo_balle[0])
135
136     return coo_balle[0]
137 # aucune coordonnée retournée si projection a échoué
138
139 # fermeture des fenetres
140 video.release()
141 cv2.destroyAllWindows()
142

```


ALGORITHMES- PYTHON RÉOLUTION

```

1  ## Modules et données
2
3  import matplotlib.pyplot as plt
4  import cv2
5  import time
6  import numpy as np
7
8  #Modèle de détection
9  from inference.models.utils import get_roboflow_model
10 model_name = "balles-de-football"
11 model_version = "1"
12 model = get_roboflow_model(model_id="{}/{}".format(model_name, model_version), api_key="WuPJ2uBzqVxnodwmTxCu")
13
14
15 # Segment de couleur
16 vert1 = np.array([0, 0, 100])
17 vert2 = np.array([100, 255, 255])
18
19 # Coordonnées des points pour la projection
20 destination = np.array([(0, 0), (0, 119), (75, 0), (75, 119)], dtype=np.float32)
21
22 # Données pour la détection des coins
23 nb_coins = 4
24 qualite = 0.1
25 aire_min = 100000
26
27 pas_de_detection = 0
28
29 ## Programmes
30
31 #génération des chemins de chaque dossier contenu dans le dossier principal à partir de la liste de leur noms
32 def nom_dossier(chemin, ldossiers):
33     return [chemin + str(dossier) for dossier in ldossiers]
34
35 #reprise de l'algorithme principal
36 def position_x(cimg):
37
38     global pas_de_detection
39     pas_de_detection = 0
40
41     frame=plt.imread(cimg)
42
43     # 1er filtre : segmentation par couleur
44     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
45     masque = cv2.inRange(hsv, vert1, vert2)
46     img_masque = cv2.bitwise_and(frame, frame, mask=masque)
47
48     # 2eme filtre : bruit
49     img_gris = cv2.cvtColor(img_masque, cv2.COLOR_BGR2GRAY)
50     noyau = np.ones((10, 10), np.uint8) # = matrice de convolution
51     seuil = cv2.threshold(img_gris, 127, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)[1]
52     seuil = cv2.morphologyEx(seuil, cv2.MORPH_CLOSE, noyau)#dilatation - erosion (fonction "closing")
53

```

```

54
55 #Recherche TERRAIN
56
57 # Image binarisée des contours (algorithme Canny)
58 bords = cv2.Canny(seuil, 100, 200)
59
60 # Listes des vecteurs et des hierarchies (liens parents-enfants) des contours dessinés
61 # Vecteur = liste de points (x,y)
62 contours, hierarchie = cv2.findContours(bords, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
63
64 c = []
65
66 for contour in contours:
67     approx = cv2.approxPolyDP(contour, 3, True) #lissage du contour (algorithme Ramer Douglas Peucker)
68     rectangle = cv2.boundingRect(approx) #cadre rectangulaire autour du contour -> approximation aire (-> filtre de taille)
69     L,h = rectangle [2], rectangle [3]
70     aire = L * h
71
72     if aire > aire_min:
73
74         enveloppe = cv2.convexHull(contour) #enveloppe convexe : plus petite enveloppe contenant tous les points du contour
75         cv2.polylines(frame, [enveloppe], True, (0, 0, 255), 2) #tracé de l'enveloppe
76
77         hauteur, largeur = bords.shape[:2]
78         enveloppe_img = np.zeros((hauteur, largeur), dtype=np.uint8)
79         cv2.drawContours(enveloppe_img, [enveloppe], 0, 255, 2) #tracé de l'enveloppe convexe
80
81         # détection des coins (algorithme Shi Tomasi)
82         distance_min = int(max(hauteur, largeur) / nb_coins)
83         coins = cv2.goodFeaturesToTrack(enveloppe_img, nb_coins, qualite, distance_min)
84         coins = np.intp(coins)
85
86         for coin in coins:
87             x, y = coin.ravel()
88             c.append((x, y))
89
90 if len(c) == nb_coins: #4
91
92     #tri des points d'intérêt par y croissants puis x croissants par paires (correspondance pour projection)
93
94     coins_tri1 = sorted(c, key = lambda x: x[0])
95
96     coins = sorted(coins_tri1[:2], key = lambda x: x[1]) + sorted(coins_tri1[2:4], key = lambda x: x[1])
97

```

```

98 | # PROJECTION
99 |
100 | source = np.array(coins, dtype=np.float32)
101 | # calcul de la matrice de projection H (3x3)
102 | H, _ = cv2.findHomography(source, destination)
103 |
104 |
105 | results = model.infer(cimg) #renvoie predictions 'confiance' décroissante -> premier set de coordonnées. Je suppose qu'
a toujours une prédiction tant que la projection est possible
106 |
107 | #(indiciage dépend de structure résultat)
108 | r = str(results[0])
109 | r2 = r.split(" ")
110 |
111 | coord = str(r2[5:9])
112 |
113 | xywh = coord[39:-1]
114 | coo_boite = xywh.split(',')
115 | x = float(coo_boite[0][2:])
116 | y = float(coo_boite[1][2:])
117 | w = float(coo_boite[2][6:])
118 | h = float(coo_boite[3][7:])
119 |
120 | coo_balle_img = np.array([x + w / 2, y + h, 1])
121 |
122 | coo_balle = np.dot(H, coo_balle_img)
123 |
124 | return coo_balle[0]
125 | #return coo_balle[1] pour la coordonnée selon la largeur
126 | pas_de_detection = 1
127 |
128 |
129 | # mesure du temps de calcul d'une position dans une image
130 | def tps_position(cimg):
131 |     t0 = time.time()
132 |     position_x(cimg)
133 |     t = time.time()
134 |     return t - t0
135 |
136 | # calcul de distance
137 | def precision(cimg): #on ne traite pas le cas sans détection car dans ce cas fonction non appelée par moyenne_dossier (L.156)
138 |
139 |     n = len(cimg)
140 |     num_image = int(cimg[n-8:n-4])
141 |     for i in range(3):
142 |         if cimg[n-8+i] == '0':
143 |             num_image = int(cimg[n-8+i:n-4]) #permet de retrouver la coordonnée attendue
144 |
145 |     #position calculée
146 |     x = position_x(cimg)
147 |
148 |     #position attendue, l'ordonnée de la balle reste la même pour 21 images
149 |     t = (num_image // 21) * 11.9
150 |
151 |     d = abs(x-t)
152 |     return d

```



```

153
154
155 #calcul du temps de traitement et de la precision moyens à une résolution fixée
156 def moyenne_dossier(cdossier,frames):
157

```

```

158     tps = 0 #somme des durées de traitement
159     d = 0 #somme des distances
160     n = 0 #nombre d'images prises en compte dans la moyenne
161

```

```

162     for i in range(1,frames):
163

```

```

164         #noms de fichiers de la forme "000i","00ij","0ijk"...
165         cimg = cdossier + '\\ ' + (4 - len(str(int(i)))) * '0' + str(int(i)) + ".jpg"
166

```

```

167         x = position_x(cimg)
168

```

```

169         if pas_de_detection == 0:
170             tps += tps_position(cimg)
171             d += precision(cimg)
172             n += 1
173

```

```

174     if n != 0:
175         return tps/n,d/n
176     return 0,0
177

```

```

178
179

```

```

180 #tracé d'un graphe à deux ordonnées (erreur, rapidité) à abscisse commune (résolution)
181 def graphe_RTP(x,y,z):
182

```

```

183     fig, axel = plt.subplots()
184     axel.set_xlabel('Résolution (% par rapport à 1920x1080)')
185

```

```

186     axel.set_ylabel('Temps de calcul (ms)', color='midnightblue')
187     axel.plot(x, y, color='midnightblue')
188     axel.tick_params(axis='y', labelcolor='midnightblue')
189

```

```

190     axel2 = axel.twinx() # second d'axe d'ordonnées qui partage un même d'axe d'abscisses
191

```

```

192     axel2.set_ylabel('Distance positions calculée - réelle (m)', color='seagreen')
193     axel2.plot(x, z, color='seagreen')
194     axel2.tick_params(axis='y', labelcolor='seagreen')
195

```

```

196     fig.tight_layout()
197

```

```

198     plt.show()
199

```

```

200 #TESTS :
201 chemin_default = "F:\\TIPE_resolution\\"
202 resolutions = [40,60,80,100,120,140,160,180,200]# Résolution de référence : 1920x1080. 20=>20% de cette résolution, etc
203 nb_img = 230
204 chemins = nom_dossier(chemin_default,resolutions) # liste des chemins des dossiers associés à diff. résolutions
205 graphe_RTP(resolutions,moyenne_dossier(chemins,nb_img)[0],moyenne_dossier(chemins,nb_img)[1])
206
207

```

ALGORITHMES- PYTHON ERREUR POSITIONS CAMÉRA

```

125 def distance(cimg):
126
127     n = len(cimg)
128     num_image = int(cimg[n-5]) #numéro de l'image étudiée pour position balle théorique
129
130     x = position_x(cimg)
131
132     if pas_de_detection == 0:
133         positions_attendues={0:0,1:59.5,2:119,3:29.75,4:59.5,5:89.25,6:0,7:59.5,8:119} #positions clefs utilisées pour tests
134         #positions_attendues={0:0,1:0,2:0,3:37.5,4:37.5,5:37.5,6:75,7:75,8:75} selon largeur
135         t = positions_attendues[num_image]
136         d = abs (x-t)
137         return d
138     return 119 #on ne "favorise" pas les positions qui ne détectent pas de terrain : distance max sur un terrain
139
140
141 def distance_moyenne_dossier(cdossier,frames,nb_position_cam):
142
143     l_distance = []
144     for i in range(1,nb_position_cam):
145
146         d = 0
147
148         for j in range(frames):
149
150             cimg = cdossier + (3 - len(str(i))) * '0'+ str(i) + '\\050' + str(int(j)) + ".jpg"
151             d += distance(cimg)
152             l_distance.append(d/frames)
153
154     return l_distance
155
156 ##Tracé graphe
157 x = [i/10 for i in range(10) for j in range(6) ] #positionnement de la caméra à partir du numéro de dossier : azimuth
158 y = [int(str(no_dossier)[-1])*pi/20 for no_dossier in range(60)] #élévation
159 z = distance_moyenne_dossier("F:\\TIPE_camera_pos\\",9,61)
160
161 fig = plt.figure()
162 ax = fig.add_subplot(projection='3d')
163
164 norme = Normalize(vmin=0, vmax=119)
165
166 triang = ax.plot_trisurf(x, y, z, cmap='RdYlGn_r', norm=norme)
167
168 ax.set_title("Distance moyenne entre les positions théoriques et issues du calcul en fonction de l'azimut et de l'élévation de la caméra")
169 ax.set_xlabel('azimut (rad)')
170 ax.set_ylabel('élévation (rad)')
171 ax.set_zlabel('distance moyenne (m)')
172
173 cbar = fig.colorbar(triang, ax=ax, shrink=0.8, aspect=10)
174 cbar.set_label('Distance moyenne (m)')
175
176 plt.show()
177 print("Distance minimale :"+str(round(min(z),3))+ "m")

```


ALGORITHMES- PYTHON POSITION CAMÉRAS

```

1 #Génération de positions de caméras pour la simulation 3d -> conversion sphériques à cartésiennes
2
3 from math import sin, cos, pi
4 import matplotlib.pyplot as plt
5 import numpy as np
6
7 def coo_sph_cart(r,t,p): #r,theta,phi
8     x = 37.5 + r * sin(p) * cos(t) #centre du terrain situé en (37.5,59.5,0)
9     y = 59.5 + r * sin(p) * sin(t)
10    z = r * cos(p)
11    return round(x,1), round(y,1), round(z,1) #on arrondit à un chiffre après virgule
12
13 n = 10
14 p = 10
15
16 #on fixe r
17 r = 300
18
19 #theta,phi en radian pour cos, sin
20 theta = [j/p for j in range(p)] # angle max condition tri points : 1 rad
21 phi = [i*(pi/2)/n for i in range(n)] #détection impossible pour t=pi/2 (=au sol)
22
23 x=[]
24 y=[]
25 z=[]
26
27 for i in range(len(theta)) :
28     for j in range(len(phi)) :
29         x.append(coo_sph_cart(r,theta[i],phi[j])[0])
30         y.append(coo_sph_cart(r,theta[i],phi[j])[1])
31         z.append(coo_sph_cart(r,theta[i],phi[j])[2])
32
33 #Représentation graphique
34
35 fig = plt.figure()
36
37 ax = plt.axes(projection='3d')
38
39 surf = ax.plot_trisurf(np.array(x), np.array(y), np.array(z), linewidth=0, antialiased=True, cmap='summer')
40 ax.set_title('Représentation visuelle des coordonnées générées')
41 plt.show()
42

```

ALGORITHMES- ARDUINO BOUCLE PRINCIPALE

```

1  #include <Stepper.h>
2  //constantes : 'const int' -> lecture uniquement (valeur non modifiable dans la boucle)
3  const int pi = 3.14159;
4  const int r = 0,005;           // rayon de la poulie (m)
5  const int k = 0.00101;        // coefficient proportionnalité terrain -> courroie
6  const int nbPasTour = 2048;
7  const int a = 2 * pi / nbPasTour; // angle d'un pas (rad)
8  const int wmax = 15.54;       // vitesse de rotation maximale (tour/min), déterminée dans le test de précision
9
10 int xi = 0;                   // initialisation position initiale
11 int xf;
12
13 int w;                         // vitesse de rotation
14
15 int Ti=0;                     // initialisation instant initial
16 int Tf;
17 int dt;                       // intervalle de temps, Tf-Ti défini dans boucle. théoriquement : dt = 1/fps
18
19 int N;                        // nombre de pas à réaliser pour une coordonnée, calculé dans la boucle
20 int PasTot = 0;              // = - nb pas à réaliser pour revenir à la position initiale (ou position actuelle en pas)
21
22 Stepper Moteur(nbPasTour,8,9,10,11);
23
24 void setup() {
25     Serial.begin(115200);
26     Serial.setTimeout(1);
27 }
28

```

ALGORITHMES- ARDUINO

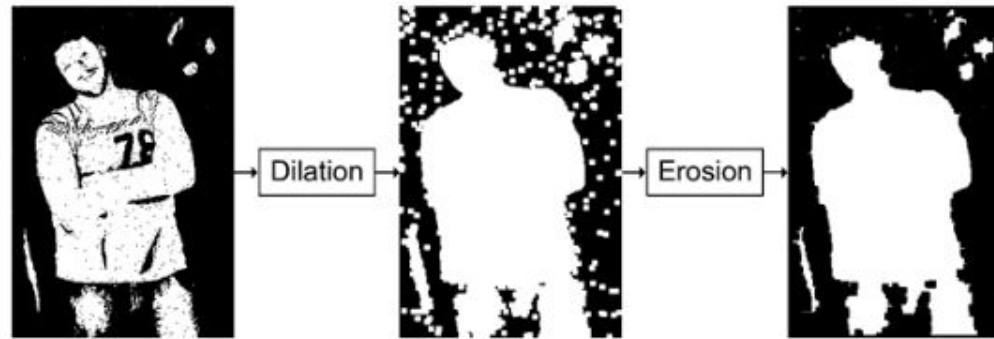
```

29 void loop() {
30   while(! Serial.available()) {
31
32     Serial.setTimeout(1000);
33
34     xf = Serial.readString().toInt();
35     Tf = millis()*0.001;      // ms -> s
36     dt = Tf-Ti;
37
38     if(xf < 120 && xf>=0){      //longueur terrain = 119m
39       w = k*(xf-xi)/(r*dt);    //vitesse
40       if(w<wmax){
41         Moteur.setSpeed(w);
42         N = k * (xf-xi) / (r * a); //nombre pas à faire
43         Moteur.step(N);
44       }
45       else{
46         Moteur.setSpeed(wmax);
47         N = k * (xf-xi) / (r * a);
48         Moteur.step(N);
49       }
50       PasTot+=N;
51       xi=xf;
52       Ti=Tf;
53     }
54   }
55
56   //retour à la position 0 si communication coupée et arrêt du programme Arduino
57   Moteur.setSpeed(1)
58   Moteur.step(-PasTot)
59   break
60 }
61

```

```
1 //détermination de la vitesse de rotation max
2 #include <Stepper.h>
3
4 const int nbpastour = 1; //en réalité : 2048 pas / tour
5 int i = 0 ;
6 int w;
7
8 Stepper Moteur(nbpastour, 8, 9, 10, 11);
9
10 void setup() {
11     Serial.begin(115200);
12     Serial.setTimeout(1);
13 }
14
15 void loop() {
16     while(i<100000){
17         w = 20480+i; //en tour/min, ici 1 tour = 1 pas donc w en pas/min (+20480 pas car vitesse max > 10 tours / min)
18         Moteur.setSpeed(w);
19         Moteur.step(100); //en pas
20
21         Serial.print(i);
22         Serial.print() ;
23         i+=1;
24         delay(200);
25     }
26 }
27 }
28
```

Dilatation, érosion



Fonction closing
© Research infinite solutions

Binarisation de l'image (seuillage)

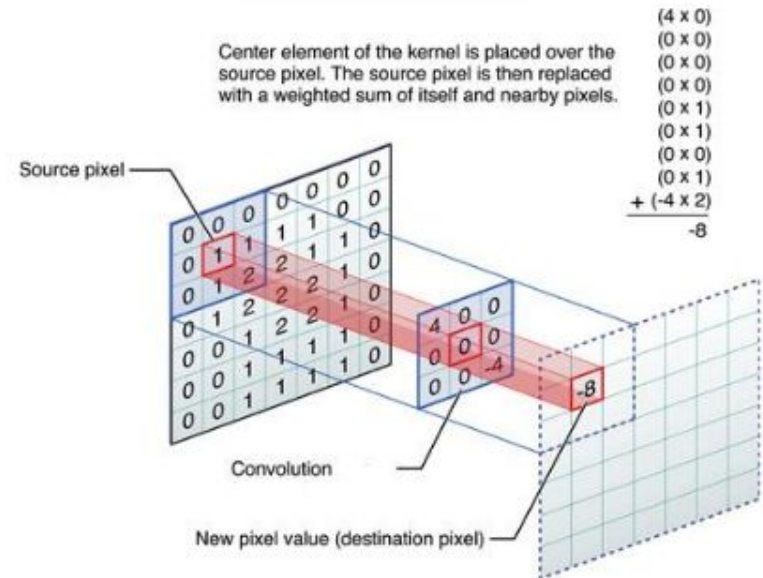
$$I_{ij} = \frac{(r + v + b)}{3} \text{ intensité du pixel } p(i,j) - \text{gris}$$

Nouvelle image $[p'(n,k)]$:

$$p'_{ij} = 255 \quad \text{si } I_{ij} \geq s \quad \text{seuil}$$

$$p'_{ij} = 0 \quad \text{sinon}$$

Convolution



Convolution

1) Filtrage **anti-bruit**

2) Produit de **convolution** avec :

$$G_x = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad ; \quad G_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

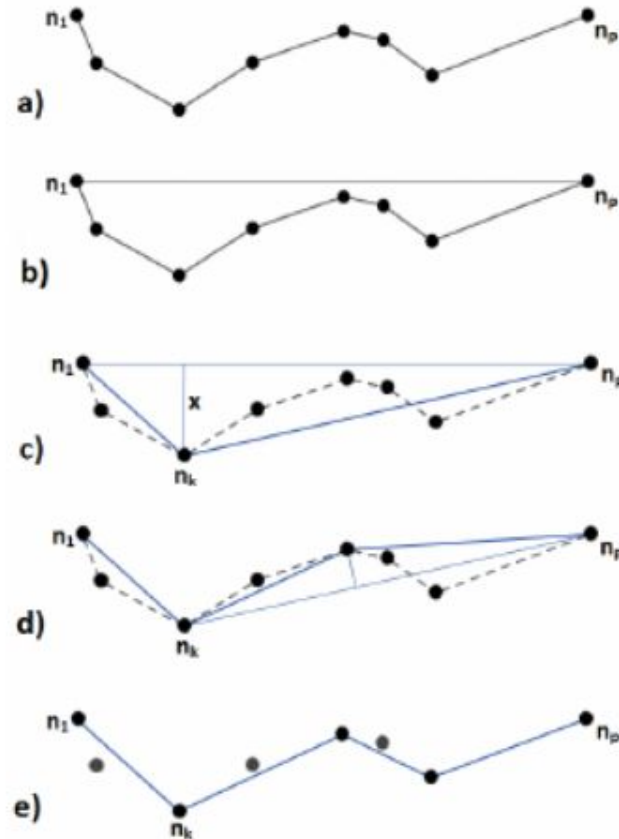
Gradient : vecteur de norme

$$|G| = \sqrt{G_x^2 + G_y^2}$$

+ => forte probabilité de présence d'un contour

3) Recherche de **maxima** locaux (seuillage)

Algorithme de Ramer Douglas Peucker

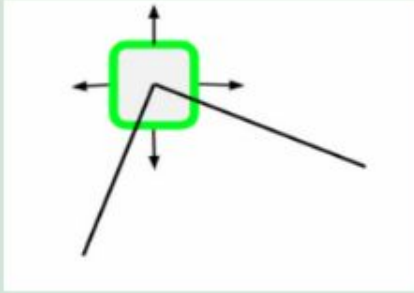


Ramer Douglas Peucker

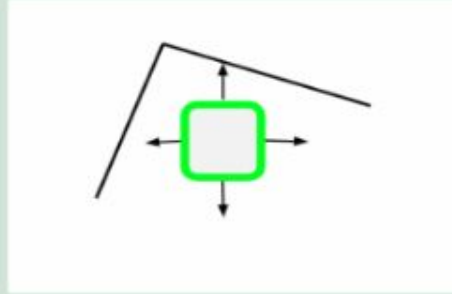
© Wojciech Mokrycki

PRINCIPE DE L'ALGORITHME - DÉTECTION DE COINS (SHI TOMASI)

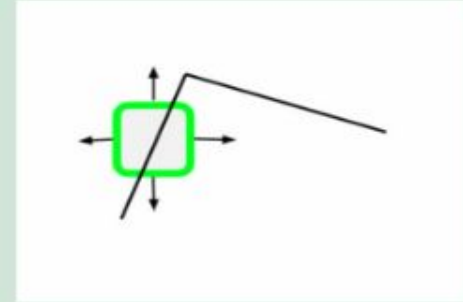
Balayage d'une "fenêtre"



Coin : changement
d'apparence dans **2+**
directions



Zone plate



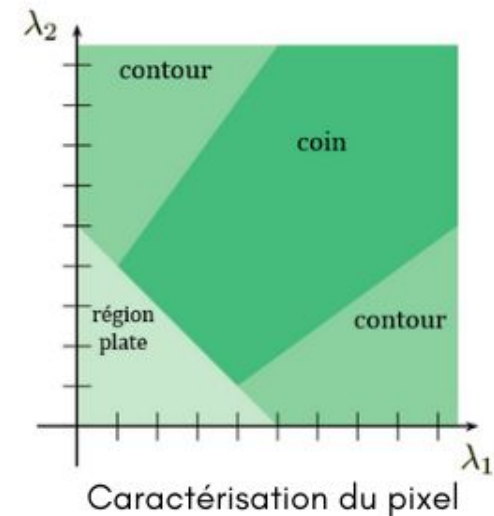
Bord : **selon 1 direction**

Recherche de coins

© Stacklima.com

- 1) **Gradient d'intensité** selon x et y
- 2) **Matrices de second moment**, M_{ij}
- 3) **Valeurs propres** de M_{ij} λ_1, λ_2 (-> courbure)

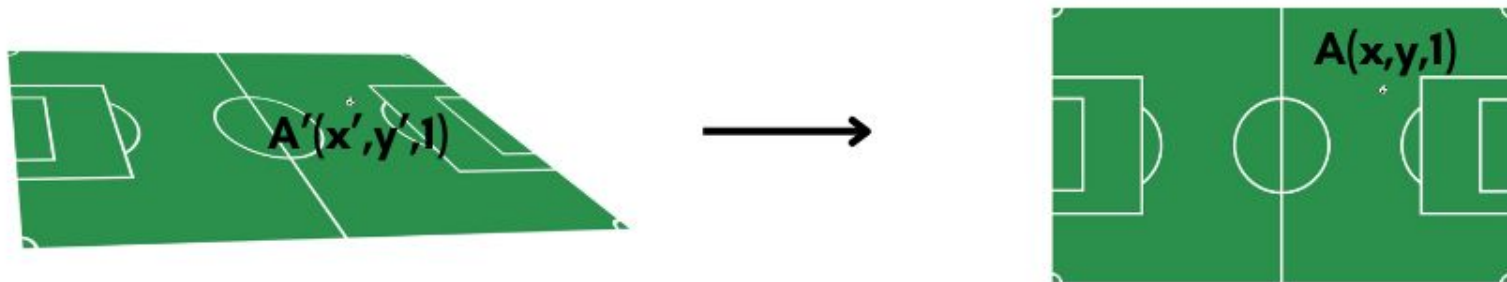
$$M_{ij} = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$



Caractérisation du pixel

© Pascal Monasse

Transformation projective plane



Si A' et A se correspondent dans les deux plans, alors il existe H définie à un scalaire près telle que :

$$H \cdot A' = A \Leftrightarrow \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

8 ddl

PRINCIPE DE L'ALGORITHME - MATRICE DE PROJECTION

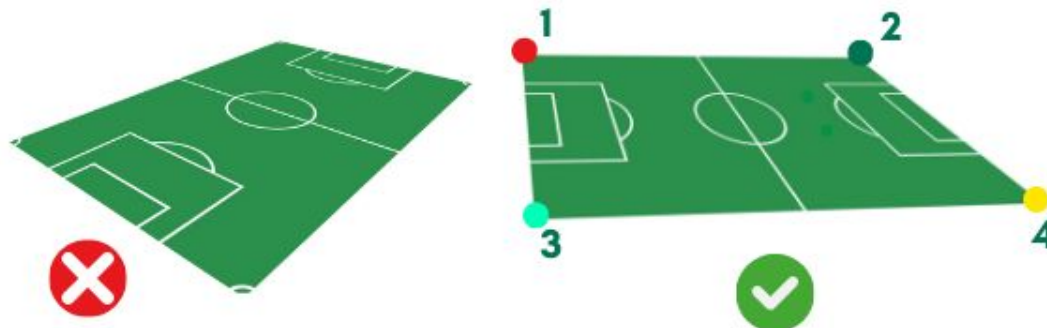
$A'(x',y')$ et $A(x,y)$ se correspondent dans les deux plans

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x'h_{11} + y'h_{12} + h_{13} \\ x'h_{21} + y'h_{22} + h_{23} \\ x'h_{31} + y'h_{32} + 1 \end{bmatrix}$$

$$\Rightarrow \begin{cases} x = \frac{x'h_{11} + y'h_{12} + h_{13}}{x'h_{31} + y'h_{32} + 1} \\ y = \frac{x'h_{21} + y'h_{22} + h_{23}}{x'h_{31} + y'h_{32} + 1} \end{cases}$$

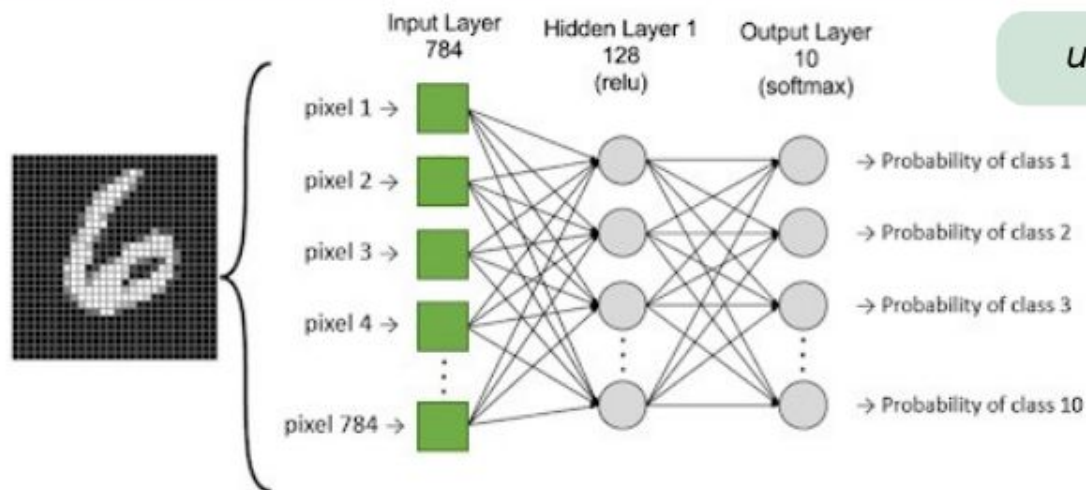
2 équations / couple de points
8 inconnues : **4 couples** pour H

- (H4) Terrain filmé tel que les sommets de la longueur opposée soient "au dessus" des autres



RÉSEAU DE NEURONES - PRINCIPE

Détection d'objet = Classification + Localisation



underfitting - overfitting

Réseau de neurones

© Datascientest.com

Feed Forward →

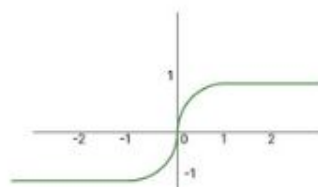
Back propagation ←



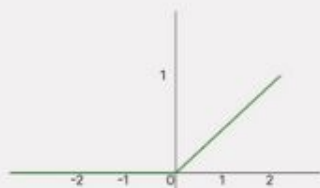
Step Function



Sigmoid



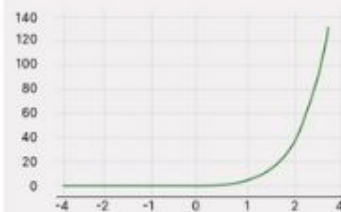
tanh



ReLU



Leaky ReLU

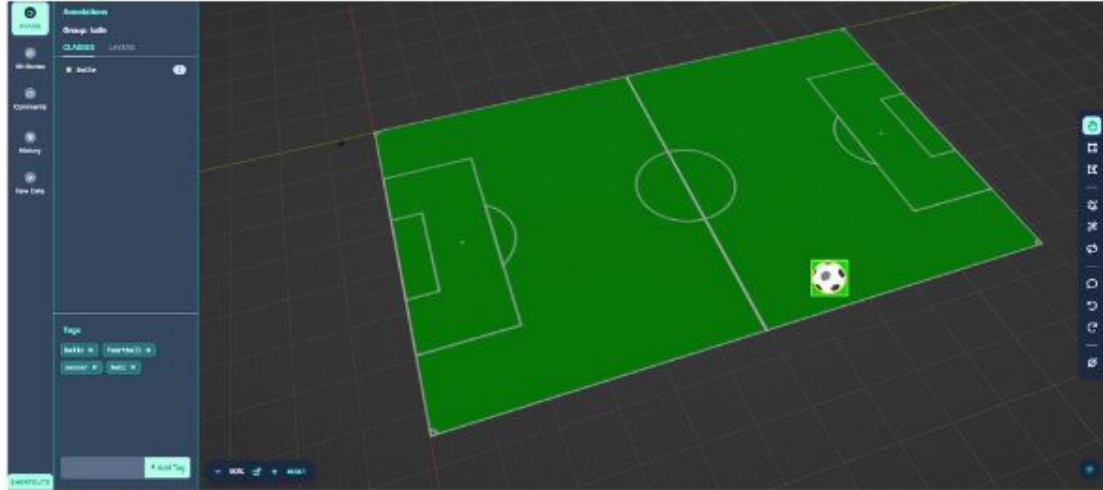


Softmax Function

Fonctions d'activation

© Shiksha.com

RÉSEAU DE NEURONES - ENTRAÎNEMENT DU MODÈLE



Interface d'annotation *Roboflow*

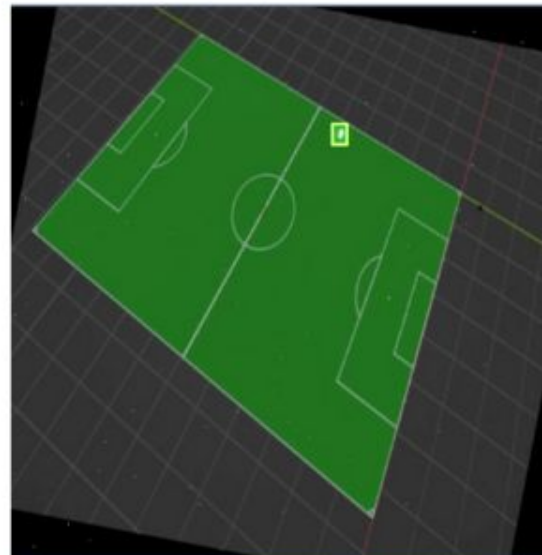


Image "augmentée"

base de données :

- Images générées par CAO (blender)
- Photos

1. Annotation images

2. Augmentation de la base de données :

- Rotation
- Exposition, Saturation
- Flou, bruit

3. Entraînement

mAP : mean Average Precision

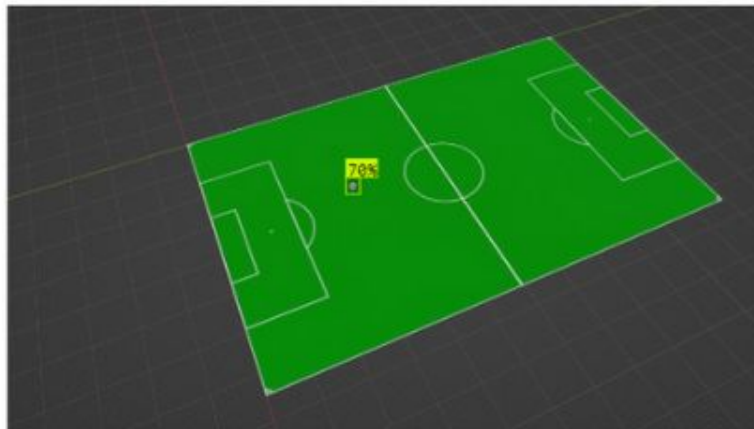
$$precision = \frac{VP}{VP + FP}$$

Boxloss localisation + dimensionnement

Classloss classe

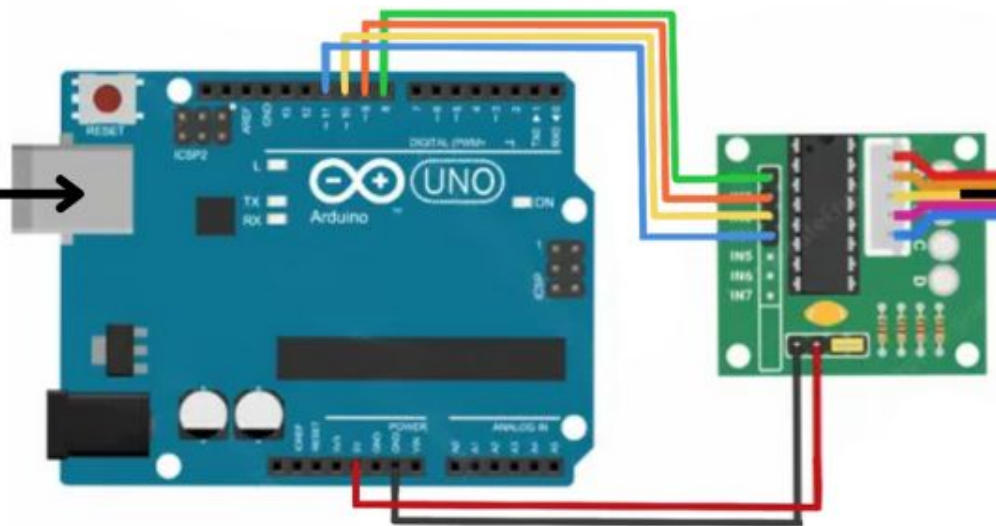
Objectloss confiance

Exemple de détection



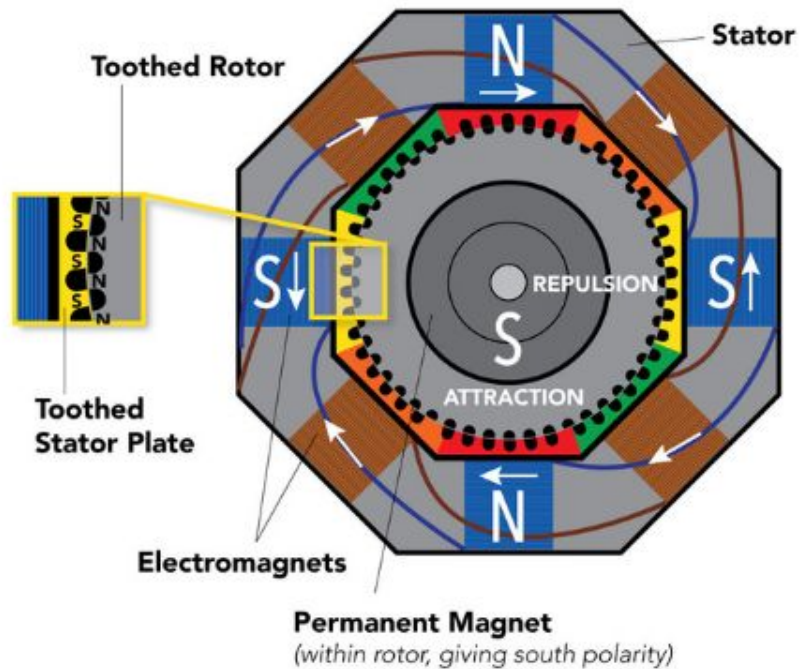
```
{
  "predictions": [
    {
      "x": 233.5,
      "y": 121.5,
      "width": 9,
      "height": 11,
      "confidence": 0.703,
      "class": "balle",
      "class_id": 0,
      "detection_id": "2b3972"
    }
  ]
}
```

Ordinateur



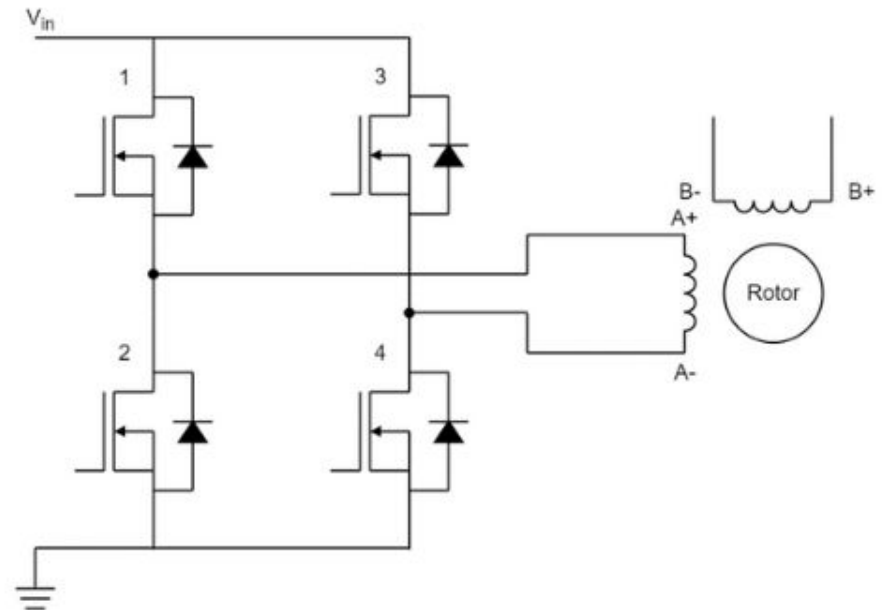
Moteur pas
à pas

ÉLECTRONIQUE - MOTEUR PAS À PAS



Moteur pas à pas

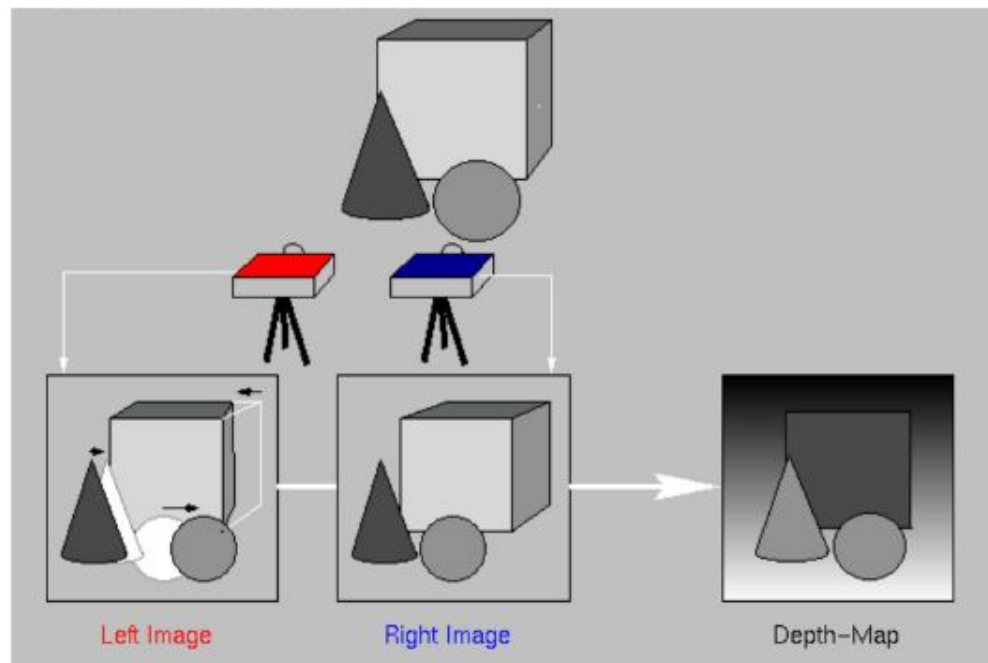
© Clippard.com



Moteur pas à pas

© Monolithic Power Systems

STÉRÉOVISION ET RECONSTITUTION 3D



Stéréovision

© Rolf Henkel



Caméra Veo

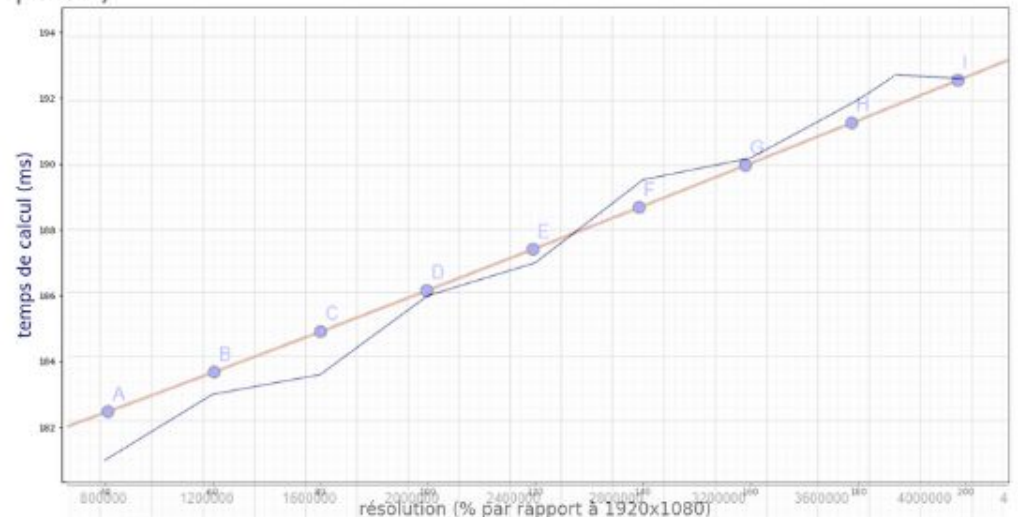
© Veo

Etude de la fonction position $x(\text{image})$:

$$o(n.p[\log(n.p)+k]+k.c.\log(c))$$

- $n.p$: taille image
- k : nombre contours détectés (à priori 1)
- c : nombre de points dans les contours

$f(n.p,k,c)$



Superposition courbes vitesse
expérimentale (Python) et fonction
complexité (Python / GeoGebra)

BIBLIOGRAPHIE

- [1] **FIFA : The incredible technology in World Cup ball** : <https://www.fifa.com/fifaplus/en/watch/weFbPlyOvE2WX4b-1hJzPw>, à partir de 42s, consulté le 17/10/23
- [2] MANON KOK, JEROEN D. HOL, THOMAS B. SCHÖN : **Using Inertial Sensors for Position and Orientation Estimation : Foundations and Trends in Signal Processing** : Vol. 11: No. 1-2 (2017), p.70-71
- [3] TAKEHIRO YAMAZAKI, MASAO SHIMIZU : **A Measuring Method for 3D Trajectory using Straightforward Triangulation** : College of Science and Technology, Nihon University : Conference draft program (2018), p.293
- [4] ASSOCIATION FOR ADVANCING AUTOMATION : **Usages pour les moteurs pas-à-pas** : <https://www.automate.org/blogs/what-kinds-of-applications-are-best-for-stepper-motors>, consulté le 21/10/23
- [5] ACADÉMIE DE LIMOGES : **Systèmes de transformation de rotation en translation** : http://pedagogie.ac-limoges.fr/sti_si/accueil/FichesConnaissances/Sequence3SSi/co/S3B22_Association_modele_composant_8.html, consulté le 23/10/23
- [6] IBM : **Apprentissage supervisé ou non supervisé** : <https://www.ibm.com/blog/supervised-vs-unsupervised-learning/> consulté le 15/11/23